Choreographic Language Extension for Markov Chain Generation

Master thesis by Jasmin Türker
Date of submission: April 14, 2026

1. Review: Dr. Adele Veschetti
2. Review: Dr. rer. nat. Richard Bubel
Darmstadt

TECHNISCHE UNIVERSITÄT DARMSTADT

Computer Science Department

Software Engineering Group

Erklärung zur Abschlussarbeit gemäß §22 Abs. 7 APB TU Darmstadt

Hiermit versichere ich, Jasmin Türker, die vorliegende Masterarbeit gemäß §22 Abs. 7 APB der TU Darmstadt ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Alle Stellen, die Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 14. April 2026

Jasmin Türker

Contents

1. Introduction **5**
- 1.1. Motivation . . . 5
- 1.2. Problem Statement and Research Objectives . . . 7
- 1.3. Contributions . . . 8
- 1.4. Structure of the Thesis . . . 9

2. Background **10**
- 2.1. Distributed Systems . . . 10
- 2.2. Choreographic Programming . . . 11
  - 2.2.1. Message-Passing Systems and the Endpoint Perspective . . . 11
  - 2.2.2. The Concept of Choreographies . . . 11
  - 2.2.3. The Choreographic Programming Paradigm . . . 12
  - 2.2.4. Endpoint Projection (EPP) and Compliance . . . 14
- 2.3. Markov Chains . . . 16
- 2.4. The PRISM Model Checker . . . 18

3. Choreography Language **20**
- 3.1. Syntax . . . 20
- 3.2. Semantics . . . 22
  - 3.2.1. Interaction and Branching . . . 22
  - 3.2.2. Conditionals and Unbalanced Flows . . . 23
  - 3.2.3. Recursive Calls . . . 24
- 3.3. Running Example: The `VendingMachine` . . . 24

4. Implementation **27**
- 4.1. Goal of the Implementation . . . 27
- 4.2. System Architecture . . . 28
- 4.3. Realization of the Theoretical Model . . . 29
  - 4.3.1. State Vector Representation . . . 31

4.3.2. Stable Node Resolution and Guard Evaluation 34
4.3.3. Synchronized Interaction and Probabilistic Branching 36
4.3.4. Recursive Control Flow and State-Space Convergence 38
4.4. Challenges 40
4.4.1. Producer–Consumer Decoupling for Transition Output 40
4.4.2. Memory Efficiency and State Deduplication 40

5. Evaluation **42**
5.1. Evaluation Setup 42
5.1.1. Threats to Validity 42
5.1.2. Methodology 43
5.2. Case Studies 44
5.2.1. Modified ThinkTeam Protocol 45
5.2.2. Peer-to-Peer Protocol 49
5.2.3. Dining Cryptographers Protocol 52

6. Related Work **56**
6.1. Choreographic Languages and Frameworks 56
6.2. Model Generation and State-Space Exploration 57
6.3. Probabilistic Verification of Distributed Systems 58

7. Conclusion and Future Work **59**

Bibliography **62**

A. Appendix **66**
A.1. Grammar 66
A.2. Vending Machine: `.sta` and `.tra` files 69
A.3. The Dining Cryptographers Protocol 70

1. Introduction

1.1. Motivation

Distributed and concurrent systems have become a fundamental pillar of modern computing. They underpin a wide range of applications, including cloud services, microservice architectures, large-scale web platforms, and Internet-of-Things (IoT) ecosystems. These systems are both *distributed*, meaning that components are deployed across multiple machines or network nodes, and *concurrent*, meaning that many processes execute simultaneously and interact with each other.

These systems are deeply integrated into everyday applications. For example, when a user logs into an online banking platform, multiple services operate concurrently and across distributed infrastructures: authentication services verify credentials, databases process account information, fraud detection mechanisms analyze behavior, and communication services ensure secure data transfer. All of these components must coordinate correctly despite running in parallel and often on different physical machines.

As these systems grow in size and complexity, coordinating many interacting components becomes increasingly difficult. Classical problems such as race conditions, deadlocks, and unintended interference between processes remain persistent challenges [6].

One main reason is that traditional programming models – such as thread-based shared-memory programming, message-passing APIs, or RPC-based service architectures – typically describe behavior only from a local perspective. Developers implement the logic of individual components or services, but the global interaction structure often remains implicit. This makes it difficult to understand how all parts of the system interact as a whole and to check whether the entire system behaves correctly. Because of subtle interactions between individual nodes, the overall system behavior is often hard to predict and may differ significantly from what one would expect based only on the behavior of single components [10].

This motivates the need for high-level abstractions that describe communication and control flow from a global perspective and support the safe design, analysis, and implementation of complex distributed systems.

Choreographic programming [27] is an emerging paradigm for describing the behavior of concurrent systems from such a global viewpoint. Instead of focusing on the internal states of individual nodes, it emphasizes coordination and communication between components. Choreographies are formal specifications of how system participants should interact. Because they are based on mathematical models, it is possible to formally prove that a system behaves as intended. Using a technique called Endpoint Projection (EPP) [18], executable implementations for each participant can be generated automatically from the global specification. These implementations are *correct-by-construction* and guarantee properties such as *deadlock-freedom by design* [18, 10].

In many practical applications, however, system behavior is not purely deterministic. Communication delays, message loss, randomized algorithms, and uncertain environments introduce probabilistic aspects that significantly influence the overall system behavior. To reason about such systems, probabilistic models such as Markov chains [23, 31] are commonly used. While these models enable quantitative analysis of properties like reliability, performance, or failure probabilities, constructing them manually for realistic distributed systems is error-prone and does not scale well. The state spaces of such models grow rapidly with the number of interacting components, making both model construction and analysis increasingly challenging.

Combining choreographic programming with probabilistic modeling offers a promising way to address these challenges. A global specification provides a compact and intuitive description of system behavior, while an automated translation into probabilistic models enables rigorous quantitative analysis. However, bridging the gap between high-level choreographic descriptions and large-scale probabilistic models raises non-trivial technical questions, particularly with respect to efficient model generation, state-space exploration, and scalability. Addressing these challenges is essential to make choreographic approaches applicable to realistic probabilistic distributed systems.

1.2. Problem Statement and Research Objectives

Problem Statement A primary example of the integration of choreographic programming and probabilistic verification is ChoreoPRISM[1], a framework designed to project global choreographies into models compatible with the PRISM model checker[2] [22]. While ChoreoPRISM establishes an important link between high-level coordination and formal quantitative analysis, it currently faces limitations in terms of linguistic expressiveness. To maintain the rigorous guarantee of *deadlock-freedom by design*, the current language often imposes strict structural constraints – such as requiring perfectly balanced conditional branches (i.e. every `if` statement must be accompanied by a corresponding `else` branch) and limiting independent internal choices by participants. In realistic distributed scenarios, however, processes frequently follow unbalanced logic or must make independent probabilistic decisions before re-synchronizing. Attempting to incorporate these features into a choreographic language is non-trivial: increasing expressiveness often breaks the formal properties that ensure the absence of deadlocks. Currently, no formal mechanism exists within ChoreoPRISM to handle these complex, "unbalanced" probabilistic flows while maintaining a provably safe projection.

This is where the principles of Applied Choreographies [18] become relevant. The challenge is not just to define new syntax, but to ensure that the Endpoint Projection (EPP) remains correct when the global model becomes more complex. Drawing from the "applied" approach, the EPP must be refined to bridge the gap between high-level probabilistic intent and the concrete, decentralized execution logic required for Markov Chain generation.

The core problem of this thesis is to extend the ChoreoPRISM language to support higher expressiveness – specifically unbalanced conditionals and independent probabilistic branching – without compromising the inherent guarantee of deadlock-freedom. This requires a fundamental update to the EPP procedure and the underlying compiler to ensure that the resulting mathematical models (DTMCs/CTMCs) remain correct-by-design.

Research Objectives To build on the strengths of ChoreoPRISM and to respond to the requirements of more realistic probabilistic modeling, this thesis defines the following goal and research questions (RQ):

[1] https://github.com/adeleveschetti/ChoreoPRISM
[2] https://www.prismmodelchecker.org/

Thesis Goal

How can the ChoreoPRISM framework be extended to support complex probabilistic logic, such as unbalanced conditionals and independent branching, while maintaining the formal guarantee of deadlock-freedom during model generation?

RQ 1: How can the choreographic language be extended to support `if-then` structures without mandatory `else` branches, and what semantic rules are required to ensure that such unbalanced flows do not lead to deadlocks after a merge?

RQ 2: How can the framework support scenarios where multiple participants make independent probabilistic choices, and how can the EPP be designed to safely manage the subsequent re-synchronization of these processes?

RQ 3: How can a compiler be implemented to translate the extended choreographic constructs into explicit-state Markov chains (DTMCs/CTMCs), and how can the resulting generating approach be evaluated with respect to structural correctness and practical feasibility?

1.3. Contributions

The primary contribution of this thesis is the extension of the ChoreoPRISM framework toward more expressive probabilistic choreographies and their direct translation into analyzable Markov chain models. In particular, this thesis makes the following contributions:

C1: Conceptual Language Extension We propose an extension of the ChoreoPRISM language to support *unbalanced conditionals* and *independent probabilistic branching*. These additions increase the expressive power of the language and enable the modeling of distributed scenarios that are difficult or impossible to represent in the original framework.

C2: EPP-Guided State-Space Exploration We develop an EPP-guided exploration approach for the direct construction of explicit-state Markov chain models from probabilistic choreographies. Rather than relying exclusively on intermediate PRISM code generation, the approach systematically derives states and transitions from the choreography and preserves probabilistic information throughout model generation.

C3: Prototype Implementation and Evaluation We implement the proposed approach in a prototype tool, ARIA, and evaluate it on several probabilistic case studies. The evaluation shows that the extended framework is capable of generating DTMC/CTMC models for non-trivial examples and provides evidence that the approach is practically feasible for the considered benchmarks.

1.4. Structure of the Thesis

This thesis is structured as follows.

Chapter 2 establishes the background for the approach presented in this thesis. It introduces distributed systems as the application context and discusses choreographic programming as a paradigm for specifying communication behavior from a global perspective. In particular, it covers message-passing systems, the endpoint perspective, choreographies, and Endpoint Projection (EPP). The chapter then presents Markov chains as the formal basis for representing probabilistic system behavior and introduces the PRISM model checker, which is used in this thesis for structural validation and comparative evaluation.

Chapter 3 formalizes the choreography language used in this thesis by defining its abstract syntax and operational semantics. For the sake of clarity and mathematical tractability, the language is presented in a process-algebraic style. This abstract representation is more suitable for formal definitions, structural reasoning, and the specification of transition rules than the concrete syntax implemented in the prototype.

Chapter 4 describes the implementation of the developed tool, including the system architecture, core data structures, and the main design decisions underlying its construction. The implementation is illustrated throughout the chapter using a running example to improve clarity and traceability.

Chapter 5 evaluates the approach using several case studies. In particular, the modified Think Team protocol, a Peer-to-Peer protocol, and the Dining Cryptographers protocol are used to demonstrate the applicability of the model generation approach.

Chapter 6 discusses related work. It reviews existing choreographic languages and frameworks, as well as approaches for model generation and state-space exploration, and positions this thesis within the existing body of research.

Finally, Chapter 7 concludes the thesis and outlines possible directions for future work.

2. Background

2.1. Distributed Systems

Distributed systems have emerged as the dominant execution environment for modern software systems [36]. They form the foundation of contemporary computing paradigms such as cloud infrastructures [2], microservice architectures [17], and Internet of Things (IoT) ecosystems [3]. While these paradigms target different application domains, they share a common architectural principle: software functionality is realized through multiple distributed components that interact and coordinate over a network in order to accomplish a common task.

According to the seminal definition by van Steen and Tanenbaum [36], a distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system. These computing elements, often referred to as nodes or endpoints [26], execute concurrently and coordinate their actions by exchanging messages via communication channels, e.g., TCP/IP sockets [7]. Despite this internal distribution and concurrency, distributed systems aim to provide distribution transparency, hiding the underlying complexity so that users and applications perceive the system as a single unified entity [36].

However, the scalability and autonomy that drive the adoption of distributed systems also introduce significant challenges. A key challenge is that, due to their autonomy, endpoints typically operate without a shared notion of time, such as a global clock [24]. This lack of synchronization, combined with the non-deterministic behavior of concurrent executions, makes programming safe distributed software notoriously error-prone [26]. Typical consequences of these complexities include safety issues such as (i) deadlocks, where systems freeze due to indefinite waiting for messages [12], and (ii) race conditions, where faulty computations occur due to uncoordinated access to shared resources [30].

The challenge of ensuring safety is further amplified by the fact that traditional diagnostic techniques, such as testing and debugging, are often unreliable in concurrent settings [5]. Bugs frequently appear non-deterministically, making them difficult to reproduce and identify [25, 28]. From a formal perspective, the concurrent execution of multiple programs leads to an exponential growth of possible execution traces, a phenomenon that complicates static analysis and verification [26]. Consequently, the correctness of a distributed system depends heavily on the precise implementation of its communication flows.

2.2. Choreographic Programming

The paradigm of *Choreographic Programming* was developed to address the inherent difficulty of coordinating separate endpoint programs. It aims to raise the level of abstraction for writing communication protocols by shifting the focus from individual node behavior to a global coordination plan, known as a *choreography* [27, 8].

2.2.1. Message-Passing Systems and the Endpoint Perspective

To understand the motivation behind choreographies, it is essential to first consider how communication is implemented in traditional distributed systems. In the message-passing paradigm, each endpoint is programmed from a local *endpoint perspective*. Communication is realized through matching pairs of primitive operations: a `send` operation at one endpoint and a corresponding `receive` operation at another [27].

As illustrated in Figure 2.1, the overall interaction emerges only implicitly from the combination of independently developed local implementations. This requires programmers to manually ensure the proper alignment of actions across all nodes. In large-scale systems, managing these dependencies becomes increasingly difficult, as a missing `send` or a misaligned `receive` can lead to communication failures or deadlocks (see Figure 2.2). Conceptually, developing systems from this perspective is *akin to* writing in assembly, where developers must manually manage low-level communication states for every participant.

2.2.2. The Concept of Choreographies

Choreographies provide a way to address these limitations by providing a global description of the interactions between endpoints, often referred to as a bird's-eye view [27]. In a

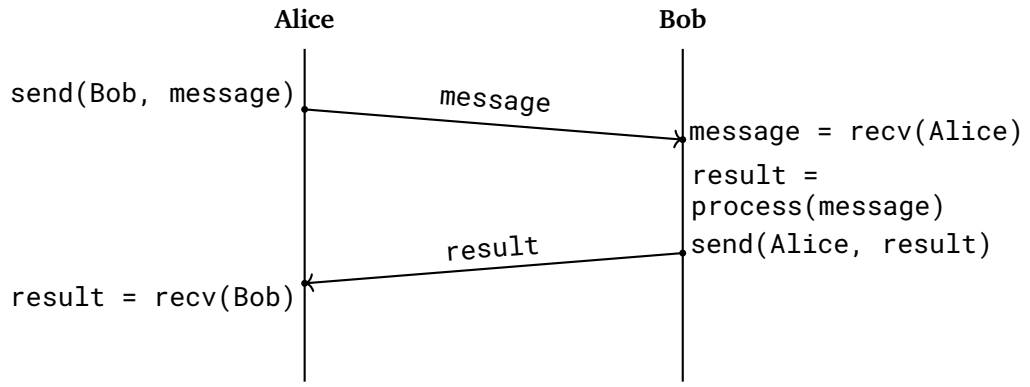


Figure 2.1.: Message Sequence Chart: Local view of communication between two participants Alice and Bob. Alice sends a message to Bob, Bob processes it locally, and then returns the result to Alice.

choreography, a communication is represented as a single *atomic interaction*, typically expressed in the "Alice and Bob" notation introduced by Needham and Schroeder [29]:

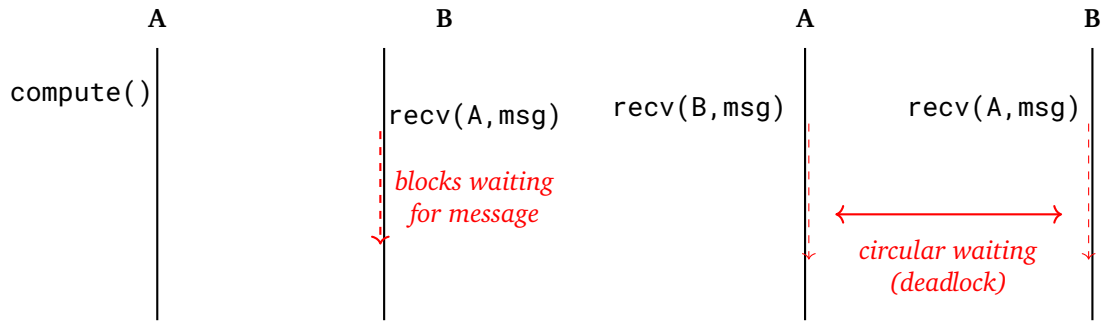
```
A.msg -> B.x
```

This notation specifies that participant `A` communicates message `msg` to participant `B`, where it is stored in variable `x`. Unlike the endpoint perspective, this interaction is described in one place, ensuring that the send and receive actions are inherently consistent.

A choreography reduces the complexity of n interacting state machines to a *single flow of discourse*. Since the early 2000s, choreographic languages have evolved from simple notations to expressive languages that include features such as local state manipulation, functions, and modularity [15, 27]. These advancements serve as the foundation for the programming paradigm discussed next.

2.2.3. The Choreographic Programming Paradigm

Choreographic Programming elevates the concept of a choreography to an executable coordination plan. Instead of implementing separate programs for each node, the developer writes a *unified, single program* for the entire system [1]. A significant advantage of this approach is that the system's logic can be *read line by line*. This eliminates the need to



(a) Blocking receive: B waits indefinitely for a message that A never sends

(b) Deadlock: both endpoints wait for each other and neither can proceed

Figure 2.2.: Typical communication failures in local message-passing systems

mentally reconstruct the protocol by *following the flow of control as it jumps from node to node*.

The most profound property of this paradigm is that choreographies are *deadlock-free by design* [8, 26]. Because communication is modeled as atomic interactions, it is structurally impossible to specify common errors like unmatched sends or receives. To illustrate this, consider the protocol for a vending machine in Listing 2.1.

```
if !VendingMachine.priceReached(amount, price) {
  User -> VendingMachine : coin
}

if VendingMachine.priceReached(amount, price) {
  if VendingMachine.inStock() {
    VendingMachine -> User : snack
  } else {
    VendingMachine -> User : refund
  }
}
```

Listing 2.1.: A minimal choreography specifying the interaction between a user and a vending machine.

In this plan, the logic of both participants is expressed within a single control flow. For instance, the second conditional is evaluated globally, which keeps the `User` and the

`VendingMachine` synchronized. Assuming that the choreography satisfies the required well-formedness conditions, projection constraints, and well-branchedness assumptions, the *else*-branch illustrates the intuition behind *deadlock-freedom by design*: since the machine is specified to send a `refund`, the `User` is projected into a corresponding state in which receiving that message is possible.

2.2.4. Endpoint Projection (EPP) and Compliance

To execute a global choreography in a distributed environment, the specification must be translated into local code for each participating node. This is achieved through *Endpoint Projection* (EPP), a formal transformation that mechanically derives a network of *compliant* endpoint programs from a global coordination plan [26]. As illustrated in Figure 2.3, this methodology shifts the burden of synchronization from the developer to the compiler toolchain, operationalizing the "bird's-eye view" of the choreography.

Formal Definition of EPP

Formally, EPP is defined as a mapping $[\![\cdot]\!]$ that projects a choreography C onto the set of process names $\mathrm{pn}(C)$. The projection of C on a specific process r, written as $[\![C]\!]_r$, returns the local process term specifying the actions r must execute. For a basic interaction, the projection operator $[\![\cdot]\!]_r$ is defined recursively:

$$[\![p \to q; C']\!]_r = \begin{cases} q!; [\![C']\!]_r & \text{if } r = p \text{ (Sender)} \\ p?; [\![C']\!]_r & \text{if } r = q \text{ (Receiver)} \\ [\![C']\!]_r & \text{if } r \notin \{p, q\} \text{ (Uninvolved)} \end{cases} \tag{2.1}$$

By assigning send primitives ($q!$) to the sender, receive primitives ($p?$) to the receiver, and skipping interactions for uninvolved parties, EPP ensures that each node only executes code strictly necessary for its role. The complete distributed system, or *network*, is then defined as the parallel composition of all individual projections: $[\![C]\!] = \prod_{p \in \mathrm{pn}(C)} p[[\![C]\!]_p]$.

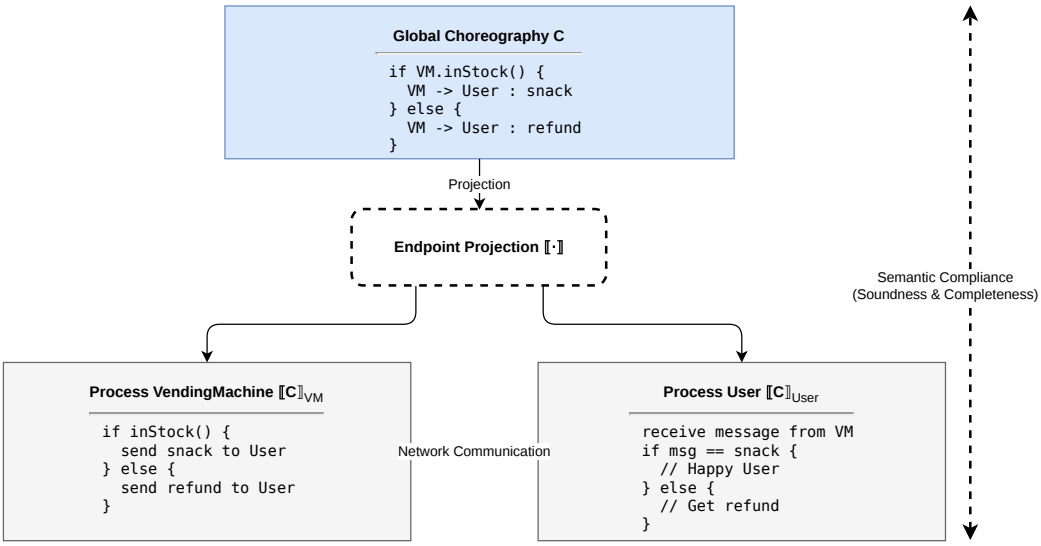


Figure 2.3.: Workflow of Endpoint Projection: Decomposing a global choreography C into compliant local terms $[\![C]\!]_p$.

Correctness and Knowledge of Choice

The central requirement for EPP is *semantic preservation*, which ensures that the generated code faithfully implements the behavior specified in the choreography. This is formally expressed through two properties [9]:

- **Completeness:** The network can perform all transitions defined in the choreography ($C \xrightarrow{\mu} C' \implies [\![C]\!] \xrightarrow{\mu} [\![C']\!]$).
- **Soundness:** The network performs *only* those actions prescribed by the choreography ($[\![C]\!] \xrightarrow{\mu} N \implies \exists C' : C \xrightarrow{\mu} C' \wedge N = [\![C']\!]$).

Here, μ represents a transition label (signifying the interaction or action being performed), while N denotes a network state, i.e., the parallel composition of process terms after a transition.

By satisfying these properties, EPP provides a formal basis for deriving endpoint implementations that are correct by construction and free from communication mismatches,

deadlocks, and race conditions under the assumptions of the model. However, a critical challenge remains the *Knowledge of Choice* (KoC) problem: when a choreography contains a conditional branch, the decision (e.g., a local check like `inStock()`) must be propagated to all affected participants to maintain global consistency [27]. EPP therefore constitutes the central mechanism that connects global choreographic specifications to local distributed behavior and provides the deterministic foundation for the extensions considered in the following chapters.

2.3. Markov Chains

To quantitatively analyze the behavior of distributed systems under uncertainty – such as communication delays, packet losses, or randomized algorithms – purely deterministic models are insufficient. Instead, this thesis utilizes Markov chains as a mathematical foundation. A Markov chain is a stochastic process that satisfies the Markov property (memorylessness): the probability of transitioning to a future state depends solely on the current state and not on the history of previously visited states [31].

In the context of probabilistic model checking and tools such as PRISM, two primary variants are distinguished: (i) Discrete-Time Markov Chains (DTMCs) and (ii) Continuous-Time Markov Chains (CTMCs).

Discrete-Time Markov Chains

Discrete-Time Markov Chains (DTMCs) model systems where transitions occur in discrete steps. They are particularly suitable for representing randomized algorithms or protocols where time is divided into fixed slots.

Formally, a DTMC [4] is defined as a tuple $\mathcal{D} = (S, s_{init}, P, L)$ where:

- S is a finite set of states;
- $s_{init} \in S$ is the initial state;
- $P : S \times S \to [0, 1]$ is the transition probability matrix, such that $\sum_{s' \in S} P(s, s') = 1$ for all $s \in S$;
- $L : S \to 2^{AP}$ is a labeling function that assigns a set of atomic propositions (AP) to each state.

Example. As a simple example, consider a message transmission attempt. From an initial state s_0, the system moves to a success state s_1 with probability 0.9 and to a failure state s_2 with probability 0.1. The states s_1 and s_2 may be modeled as absorbing states, i.e., once reached, the system remains in them with probability 1. Formally, for the state order (s_0, s_1, s_2), the DTMC is given by the transition matrix

$$P = \begin{pmatrix} 0 & 0.9 & 0.1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

This illustrates the characteristic feature of a DTMC: the next state is chosen according to a discrete probability distribution over successor states.

In the context of this thesis, DTMCs serve as the semantic foundation to model the operational behavior of probabilistic choreographies. Within this framework, states represent global system configurations, while transitions capture atomic execution steps, such as message exchanges or probabilistic branching decisions. The transition probabilities are directly induced by the weights assigned to probabilistic choices within the choreographic specification. Consequently, DTMCs provide a rigorous basis for the quantitative analysis of protocols characterized by discrete-time behavior, such as the Peer-to-Peer protocol (Section 5.2.2) and the Dining Cryptographers protocol (Section 5.2.3).

Continuous-Time Markov Chains

While DTMCs are suitable for discrete steps, many distributed systems exhibit behavior that depends on continuous time, such as communication latencies or hardware failure rates. For these scenarios, Continuous-Time Markov Chains (CTMCs) are employed.

A CTMC [4] is formally defined as a tuple $\mathcal{C} = (S, s_{init}, R, L)$, where S, s_{init}, and L are defined as in a DTMC, but:

- $R : S \times S \to \mathbb{R}_{\geq 0}$ is the transition rate matrix.

The time spent in a state s before a transition occurs follows an exponential distribution with the rate $E(s) = \sum_{s' \in S} R(s, s')$.

Example. As a simple example, consider a server that alternates between being idle and processing requests. Let s_0 denote the idle state, s_1 the processing state, and s_2 a failure state. From s_0, the server starts processing a request with rate λ. From s_1, it either completes the request and returns to s_0 with rate μ, or it fails and moves to s_2 with rate θ. For the state order (s_0, s_1, s_2), this yields the rate matrix

$$R = \begin{pmatrix} 0 & \lambda & 0 \\ \mu & 0 & \theta \\ 0 & 0 & 0 \end{pmatrix}.$$

This illustrates the defining feature of a CTMC: transitions are governed by rates, which determine how quickly competing events occur in continuous time.

In contrast to DTMCs, CTMCs capture systems whose behavior depends not only on probabilistic branching, but also on the timing of events. This is particularly relevant for modeling real-time aspects of distributed systems, such as communication latencies or stochastic delays. In this thesis, CTMCs are used to analyze time- and performance-related properties, as demonstrated in the evaluation of the Modified Think Team Protocol (Section 5.2.1).

In summary, DTMCs model probabilistic branching in discrete execution steps, whereas CTMCs capture stochastic behavior in continuous time through transition rates.

2.4. The PRISM Model Checker

To evaluate the efficiency and correctness of the choreographic language extension developed in this thesis, the PRISM model checker [23] is used as a primary reference and benchmarking tool. PRISM is a widely established framework for Probabilistic Model Checking (PMC), a formal verification technique used to analyze the likelihood of specific behaviors in stochastic systems. In this work, PRISM serves as a ”gold standard” to validate that the state spaces generated by the choreographic projection are mathematically accurate and to provide a baseline for performance comparisons.

PRISM typically operates on high-level system descriptions written in its native PRISM modeling language. This state-based language organizes systems into modules that interact via shared variables and synchronized actions. When a user provides a PRISM model (e.g., a .pm file), the tool performs a process called state-space construction:

it explores all possible configurations and transitions to build a global mathematical representation, such as a Discrete-Time Markov Chain (DTMC).

A central part of the evaluation in this thesis (see Chapter 5) involves comparing the efficiency of this construction process. Specifically, we benchmark the runtime required by PRISM's native compiler to build a model against the time taken by our direct generation approach from choreographic specifications.

While PRISM is primarily used through its high-level language, it also provides an explicit interface for pre-constructed models. This interface allows external tools to bypass the PRISM compiler and provide the transition matrix directly. The explicit representation consists of three primary file formats:

- **States Files (`.sta`):** An enumeration of all reachable global states.
- **Transitions Files (`.tra`):** A list of all transitions between states, including their associated probabilities (for DTMCs) or rates (for CTMCs).
- **Labels Files (`.lab`):** A mapping of states to atomic propositions for property verification.

The tool developed in this work is designed to generate these explicit formats directly. This enables a structural validation: by comparing the number of states and transitions in the .sta and .tra files generated by our tool with those generated by PRISM from an equivalent manual model, we can formally verify the correctness of the choreographic projection logic.

Although the core contribution of this thesis is the generation of models, the ultimate purpose of these models is their analysis. PRISM enables the verification of complex properties expressed in temporal logics, such as Probabilistic Computation Tree Logic (PCTL) for DTMCs and Continuous Stochastic Logic (CSL) for CTMCs. These logics allow for the calculation of:

- *Probabilities:* "What is the likelihood that the protocol terminates within N steps?"
- *Expected Rewards:* "What is the expected resource consumption (e.g., energy or time) until termination?"

By ensuring that our tool outputs PRISM-compatible explicit files, we provide a seamless path for users to utilize PRISM's powerful numerical solvers for the downstream verification of generated choreographies.

3. Choreography Language

The purpose of this chapter is to formalize the syntax and semantics of the choreography language considered within the scope of this thesis. For the sake of clarity and mathematical tractability, we adopt a process-algebraic format, which makes the notation more concise and better suited for formal definitions, structural induction, and correctness proofs. This abstract representation departs slightly from the concrete syntax implemented in our tool, allowing us to focus on the underlying coordination logic and its direct mapping to probabilistic state-and-transition models.

This formalization builds upon the core framework of ChoreoPRISM, as introduced by Carbone and Veschetti [10]. While we maintain the fundamental principle of global coordination, our model introduces significant extensions to handle unbalanced control flows and independent probabilistic branching. These features allow for a more flexible description of distributed protocols while enabling a direct mapping to the state-and-transition representation required for Markov chain generation.

In the following sections, we first define the abstract syntax of the language, followed by an operational semantics that describes how a choreography evolves alongside the system's global state. (Finally, we demonstrate how this semantic model serves as the basis for the automated generation of PRISM-compatible explicit models.)

3.1. Syntax

The abstract syntax of the choreography language is formally defined using the Backus-Naur Form (BNF).

Let p, q range over a set of participants $\mathcal{P}$, x over a set of variables *Var*, and v over a set of values *Val*.

The core components of the language, choreographies C, are defined as follows:

$$
\begin{array}{rcll}
C & ::= & p \to \{p_1, \ldots, p_n\} : \sum_{j \in J} \lambda_j : u_j; C_j & \text{(Interaction)} \\
 & | & \text{if } E@p \text{ then } C_1 \text{ [else } C_2] & \text{(Conditional)} \\
 & | & X & \text{(Recursive call)} \\
 & | & \mathbf{0} & \text{(Inactivity)} \\
\mathcal{D} & ::= & X \stackrel{\text{def}}{=} C, \mathcal{D} \mid \emptyset & \text{(Definitions)} \\
u & ::= & (x' = E) \,\&\, u \mid (x' = E) & \text{(Assignments)} \\
E, g & ::= & f(\tilde{E}) \mid x \mid v & \text{(Expressions)}
\end{array}
\tag{3.1}
$$

Syntactic Constructs and Extensions Building on the foundation of Carbone and Veschetti [10], the syntax is adapted to support the additional requirements of the probabilistic models considered in this work.

- **Probabilistic Interaction:** The interaction term $p \to \{p_1, \ldots, p_n\} : \sum_{j \in J} \lambda_j : u_j; C_j$ describes an interaction initiated by participant p towards a set of receivers. The mapping to Markov Chain transitions is performed by interpreting each index $j \in J$ as a distinct outgoing edge from the current configuration (S, C). Specifically, for every branch j, the operational semantics (see Section 3.2) defines a transition to a successor configuration $(S[u_j], C_j)$ with an associated label λ_j. The term u_j denotes a set of local variable updates or assignments performed by the participants during the interaction. In the generated model, the labels λ_j are directly mapped to the transition rates for CTMCs or probabilities for DTMCs. This approach realizes the direct generation of the underlying stochastic model, a strategy suggested in [10] to improve performance by bypassing intermediate code projection.
- **Unbalanced Conditionals:** A central contribution of this work is the support for unbalanced conditionals. In our syntax, the `else` branch is optional. If omitted, it is semantically equivalent to the null choreography **0**. This allows for the modeling of "fire-and-forget" or "trigger-based" logic where certain participants only act if a specific condition is met, without forcing a synchronized alternative path.
- **Assignments and State Updates:** The term u allows for multiple simultaneous state updates using the & operator. This follows the PRISM notation $x' = E$, indicating that variable x will take the value of expression E in the next state of the generated Markov Chain.

3.2. Semantics

The semantics is adapted from [10] and extended to support probabilistic branching and unbalanced conditionals.

The operational semantics forms the core of this thesis, as it provides a formal specification of the rules governing state transitions in the system. Beyond its theoretical role, the semantics serves as the foundation for the implementation of our compiler (see Chapter 4). By directly reflecting these formal rules, the tool systematically explores the state space of a choreography and derives the corresponding transitions of the generated Markov chain in a mathematically sound manner.

A global state S is defined as a mapping from variables to values: $S : \texttt{Var} \to \texttt{Val}$. We define the semantics as the smallest relation $\longrightarrow^{\mathcal{D}} \subseteq (S, C) \times \mathbb{R} \times (S, C)$. A transition $(S, C) \longrightarrow_{\lambda} (S^{'}, C^{'})$ denotes that the system moves from state S and choreography C to the new state $S^{'}$ and (residual) choreography $C^{'}$ with rate or probability λ.

3.2.1. Interaction and Branching

The (Interact) rule models the synchronous communication between a single sender p and a set of receivers $p_1, ..., p_n$. In the global view, this interaction is an atomic step, meaning that all involved participants synchronize and transition simultaneously. The branching behavior is inherently stochastic: the interaction offers multiple possible outcomes (branches), indexed by J. Each branch $k \in J$ is associated with a specific stochastic value λ_k, which determines the likelihood or timing of that branch being selected. The merge behavior is handled structurally by the continuations C_k: after the synchronization and the execution of local updates u_k, the system evolves into the subsequent choreography C_k. If different branches lead to the same continuation term, they effectively merge in the resulting state space.

$$\textbf{(Interact)} \quad \frac{k \in J}{(S, p \to \{p_1, \ldots, p_n\} : \sum_{j \in J} \lambda_j : u_j; C_j) \longrightarrow \lambda_k (S[u_k], C_k)} \tag{3.2}$$

In this rule, J denotes a finite set of indices representing the possible outcomes of the interaction. For each specific branch $k \in J$, the term λ_k represents the stochastic parameter of the transition. To ensure a well-defined mapping to Markovian models, we distinguish between two semantics:

- In a **Discrete-Time Markov Chain (DTMC)**, λ_k denotes a **transition probability**, where $0 \leq \lambda_k \leq 1$ and the sum of all branch probabilities must satisfy $\sum_{j \in J} \lambda_j = 1$.
- In a **Continuous-Time Markov Chain (CTMC)**, λ_k denotes a **transition rate**, representing the parameter of the exponential distribution governing the time until the interaction occurs.

3.2.2. Conditionals and Unbalanced Flows

Conditionals represent local decision points determined by a specific participant p. The (IfThenElse) rule captures deterministic branching based on the evaluation of a local guard E in the current state $S(E \downarrow_S)$.

$$\textbf{(IfThenElse-True)} \quad \frac{E \downarrow_S = \text{tt}}{(S, \text{if } E@p \text{ then } C_1 \text{ else } C_2) \longrightarrow_1 (S, C_1)} \tag{3.3}$$

$$\textbf{(IfThenElse-False-Balanced)} \quad \frac{E \downarrow_S = \text{ff}}{(S, \text{if } E@p \text{ then } C_1 \text{ else } C_2) \longrightarrow_1 (S, C_2)} \tag{3.4}$$

A significant extension in our framework is the support for unbalanced conditionals, i.e. an `if`-branch without a corresponding `else`-branch. This flexibility is crucial for modeling *trigger-based* protocols where certain participants only act if a specific condition is met, without necessitating a synchronized alternative path. Formally, if the guard evaluates to `false` and no alternative branch exists, the choreography terminates.

$$\textbf{(IfThenElse-False-Unbalanced)} \quad \frac{E \downarrow_S = \text{ff}}{(S, \text{if } E@p \text{ then } C_1) \longrightarrow_1 (S, \mathbf{0})} \tag{3.5}$$

In this case, the system transitions deterministically to the inactive state **0**, representing the completion of the local execution flow.

3.2.3. Recursive Calls

To allow for the modeling of repetitive or infinite behaviors – such as randomized retries, polling loops, or long-running protocols – the language supports recursion through procedure calls. This mechanism enables a choreography to jump back to a previously defined point in the execution flow.

$$\textbf{(Call)} \quad \frac{X \stackrel{\text{def}}{=} C \in \mathcal{D}}{(S, X) \longrightarrow_1 (S, C)} \tag{3.6}$$

In this rule X denotes a unique protocol identifier, and $\mathcal{D}$ represents the global set of definitions. The notation $X \stackrel{\text{def}}{=} C$ indicates that the identifier X is formally defined as the choreography C within this set.

From a semantic perspective the (Call) rule describes the unfolding of the identifier: the system transitions from the call X to its corresponding body C while maintaining the current global state S. Since this is a structural step in the control flow, it is treated as a deterministic transition with a weight of 1.

3.3. Running Example: The `VendingMachine`

In Section 2.2.3 we introduced a vending machine protocol as an informal example (see Listing 2.1) to illustrate the concept of choreographic coordination. We now revisit the scenario using the formal abstract syntax and operational semantics defined in this chapter.

The protocol models the repeated insertion of coins until the required price is reached, followed by a probabilistic payout step. This example demonstrates how recursive calls, conditional branching, and probabilistic interactions are represented in the formal syntax and how they induce a corresponding Markov chain.

The informal protocol can now be expressed in the abstract syntax introduced in Section 3.1. The resulting choreography is given in Listing 3.1.

Based on the operational semantics defined in Section 3.2, the choreography induces a corresponding Markov chain. Each state of the Markov chain is represented as a pair (S, C), where S denotes the valuation of variables and C the remaining choreography. For

```
{
  Start :=
  if "money < price" @user then {
    ["1.0"] "money'=money+1" @user .
    Start
  } else {
    Payout
  }

  Payout :=
  if "money >= price" @machine then {
    machine -> user : (
    + ["0.9"] "snack'=1, money'=0" . END
    + ["0.1"] "snack'=0, change'=1, money'=0" . END
    )
  }
}
```

Listing 3.1.: A running example: `VendingMachine`

illustration, we consider a concrete instance with $price = 2$, resulting in the state space shown in Figure 3.1.

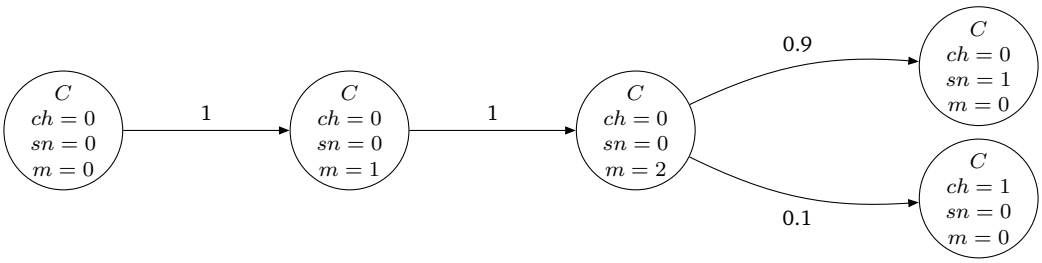


Figure 3.1.: Representation of the induced Markov Chain

To make the connection between the operational semantics and the Markov chain in Figure 3.1 explicit, we sketch the derivation of the first transitions for the concrete instance with $price = 2$. Let the initial valuation be

$$S_0 = \{money \mapsto 0,\ snack \mapsto 0,\ change \mapsto 0,\ price \mapsto 2\}.$$

The initial configuration is then given by

$$(S_0,\ Start).$$

Since the guard $money < price$ evaluates to true in S_0, the semantics selects the then-branch of the conditional in `Start`. Executing the assignment

$$\text{["1.0"]}\ money' = money + 1\ @user$$

updates the valuation to

$$S_1 = \{money \mapsto 1,\ snack \mapsto 0,\ change \mapsto 0,\ price \mapsto 2\},$$

and the recursive call returns control to `Start`. Hence, one obtains the transition

$$(S_0,\ Start) \longrightarrow (S_1,\ Start).$$

Applying the same rule once more yields

$$(S_1,\ Start) \longrightarrow (S_2,\ Payout),$$

where

$$S_2 = \{money \mapsto 2,\ snack \mapsto 0,\ change \mapsto 0,\ price \mapsto 2\}.$$

At this point, the guard $money < price$ is false, so the else-branch is selected, which transfers control to `Payout`.

In the configuration $(S_2,\ Payout)$, the guard $money \geq price$ evaluates to true, enabling the probabilistic interaction at the machine. According to the semantics of probabilistic branching, this yields two successor configurations:

$$(S_2,\ Payout) \xrightarrow{0.9} (S_3,\ END) \qquad \text{and} \qquad (S_2,\ Payout) \xrightarrow{0.1} (S_4,\ END),$$

where

$$S_3 = \{money \mapsto 0,\ snack \mapsto 1,\ change \mapsto 0,\ price \mapsto 2\}$$

and

$$S_4 = \{money \mapsto 0,\ snack \mapsto 0,\ change \mapsto 1,\ price \mapsto 2\}.$$

These derivation steps correspond directly to the states and transitions shown in Figure 3.1: the first two transitions are deterministic and therefore labeled with probability 1, whereas the final payout step induces the probabilistic branching with probabilities 0.9 and 0.1.

4. Implementation

This chapter describes how we realized the formal framework developed in Chapter 3 into an executable prototype. To this end, we present ARIA[1] (Automated Reasoning for Interactive Choreographies), a tool that systematically explores choreographic executions and derives the corresponding Markov chain in an automated and efficient manner.

4.1. Goal of the Implementation

The goal of the implementation is to make the formal framework established in Chapter 3 practically usable. While the theoretical model shows that choreographies give rise to Markov chains, applying this construction to realistic systems requires an automated exploration procedure that can systematically capture large and complex state spaces. We address this challenge with ARIA by providing an automated pipeline from high-level choreographic specifications to stochastic models.

Specifically, we designed the implementation to achieve the following four objectives:

1. **Automated State-Space Exploration:** We aim to explore all reachable configurations (S, C) of a choreography in a systematic manner. To this end, we implemented the operational semantics using a worklist-based exploration algorithm, which allows us to construct the reachable state space automatically.

2. **Direct Model Generation:** We focus on the direct construction of the state-space from the operational semantics. By generating states and transitions without relying on an intermediate high-level modeling language, we ensure a precise mapping between our formal model and the resulting stochastic process. This approach allows us to avoid the overhead of an additional compilation step and the necessity

[1] The complete source code of ARIA is available at: `https://github.com/JTuerker/ARIA`

of encoding complex, unbalanced control flow into the restricted syntax of guarded command languages.

3. **Semantic Fidelity:** We ensure that the implementation remains closely aligned with the formal semantics. We designed ARIA such that each generated transition corresponds to a valid application of the inference rules defined in Section 3.2. This ensures that the deadlock-freedom-by-design guarantees of our model are preserved in the generated output.

4. **Interoperability and Validation:** Finally, we ensure that the generated model is fully compatible with established verification tools. We achieve this by exporting the state space into the PRISM explicit format, specifically using the `.sta` (state) and `.tra` (transition) file extensions. This interoperability allows us to use ARIA as a specialized front-end for choreographies while leveraging the PRISM model checker for the subsequent analysis and validation.

4.2. System Architecture

We implemented ARIA as a standalone Java application. Its architecture follows a staged pipeline consisting of a *front-end* for choreography processing, a *back-end* for state-space exploration, and a *post-processing* step for serialization to model-checker formats. Java was chosen as the implementation language because it offers a practical balance between portability, type safety, and performance for a compiler-like analysis tool. The project is managed with MAVEN to support consistent dependency management and reproducible builds.

For the *front-end*, we utilize ANTLR4 [32] as a parser generator to process choreography specifications. Based on the context-free grammar provided by Carbone and Veschetti [10] (see Appendix A.1), we employ ANTLR4 to generate the lexer and parser. We then recursively traverse the resulting parse tree with a custom visitor to construct an Abstract Syntax Tree (AST). This AST serves as our internal representation of the input choreography for the subsequent state-space exploration.

The back-end of ARIA is centered around a worklist-based state-space exploration engine. We chose an iterative approach over a recursive implementation for two reasons. First, it avoids Java's call-stack limitations when exploring deep state spaces. Second, it decouples successor generation from the exploration order, allowing the scheduling policy to be

varied independently of the semantic rules. This design proved essential for handling state-space explosion. The iterative worklist-based design decouples successor generation from the exploration order. This flexibility allows the engine to employ a *biased hybrid search strategy*, which can be tuned to balance between different traversal policies depending on the model's complexity (see Section 4.4.2).

Starting from an initial configuration (S, C), the exploration engine incrementally computes successor states by applying the operational semantics defined in Chapter 3. To avoid redundant exploration and ensure termination, visited configurations are stored in a centralized hash-based data structure for efficient state deduplication. This is essential for explicit-state generation, since the same global configuration may be reached through different concurrent execution paths. To further optimize performance-critical operations, we employ specialized collection classes from the FastUtil library. By using custom hashing strategies and avoiding the overhead of standard boxed collection types, we reduce heap usage and accelerate state lookups, especially for large state vectors where equality must be determined by content rather than object identity.

To interface with probabilistic model checkers, each discovered configuration is assigned a unique numerical identifier. Serialization into the PRISM explicit format (`.sta` and `.tra`) is performed as a separate post-processing step after exploration has concluded. This separation ensures that the available memory is primarily dedicated to state-space exploration, without additional overhead from I/O operations and string formatting during the search phase.

Finally, the implementation is maintained in a Git-based version control system in order to support traceability and reproducibility of further development.

4.3. Realization of the Theoretical Model

This section describes the realization of the theoretical model, bridging the gap between formal semantics and technical execution. To provide a coherent guide through the implementation details, we again use the `VendingMachine` protocol as a running example, this time in a recursive variant that models continuous operation (see Listing 4.1).

Lines 13 and 14 realize the recursive continuation of the protocol: after completing the `Payout` phase, both probabilistic outcomes transition back to the `Start` protocol, allowing the process to re-enter its initial waiting state and handle further user inputs

```
preamble
"ctmc";
"const int price = 2;"
endpreamble;

user -> user : "money=0";
machine -> machine : "snack=0", "change=0";

{
  Start :=
  if "money < price" @user then {
    ["1.0"] "money'=money+1" @user .
    Start
  } else {
    Payout
  }

  Payout :=
  if "money >= price" @machine then {
    machine -> user : (
    + ["0.9"] "snack'=1, money'=0" . Start
    + ["0.1"] "snack'=0, change'=1, money'=0" . Start
    )
  }
}
```

Listing 4.1.: The recursive `VendingMachine` protocol

indefinitely. From a technical perspective, this recursive definition implies that the induced state-space representation contains cycles rather than terminating after a single interaction. This provides a critical test case for our compiler: the exploration engine must correctly identify previously visited state vectors and avoid re-expanding them indefinitely. Only then can the exploration terminate once the finite set of reachable states has been exhausted.

To bridge the gap between this textual specification and the automated generation of the states and transitions, the compiler first translates the input protocol into an Abstract Syntax Tree (AST). As shown in Figure 4.1, this hierarchical representation breaks down the high-level syntax into structured components, including the preamble, role definitions, and control-flow nodes.

The AST representation is fundamental for the exploration engine: each node within the

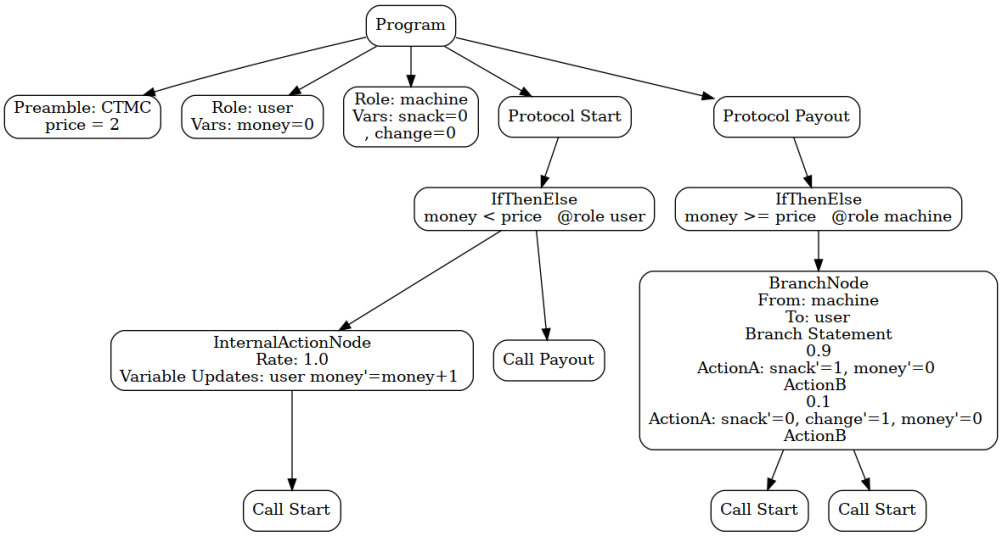


Figure 4.1.: AST representation of the Vending Machine protocol: each node within the tree is assigned a unique numeric identifier (`pcId`) during the compilation process. These IDs allow the engine to track the progress of each process by storing the current node's identifier in the program counter segment of the state vector.

tree is assigned a unique numeric identifier (`pcId`) during the compilation process (see Section 4.3.1).

4.3.1. State Vector Representation

To manage the state efficiently, we represent the abstract state S as a flat `Object[]` array in memory. We partition this vector into two distinct logical segments: the *program counter* segment and the *variable* segment.

The Program Counter (PC) segment stores the current control location for each active process. We manage the mapping of process names to their respective vector indices using the `processNameToPcIndex` map. During initialization, we define the boundary for the following segment by setting the `variableSectionStart` offset, which corresponds to the total number of processes in the system (see Listing 4.2).

```java

    public MatrixBuilderOnTheFlyWithStateVectors(ProgramNode astRoot) {

        int pcIndexCounter = 0;
        for (String processName : processNames) {
            this.processNameToPcIndex.put(processName, pcIndexCounter++);
        }
        this.variableSectionStart = pcIndexCounter;

        Set<Node> allInstructionNodes = new HashSet<>();
        collectAllInstructionNodes(astRoot, allInstructionNodes);
        int pcIdCounter = 0;
        for (Node instructionNode : allInstructionNodes) {
            this.nodeToPcId.put(instructionNode, pcIdCounter);
            this.pcIdToNode.add(instructionNode);
            pcIdCounter++;
        }

    }

```

Listing 4.2.: Initialization of the Program Counter mapping

A key characteristic of our implementation is the dynamic assignment of numeric IDs to AST nodes. Unlike static instruction pointers, these IDs are generated at runtime during each exploration run. Importantly, the IDs do not identify the type of a statement, but its concrete position in the program tree. Thus, even if two syntactically identical statements appear consecutively in the choreography, each receives its own `pcId`, because they correspond to different locations in the AST.

To support this encoding, we maintain a bidirectional mapping. The `nodeToPcId` map translates AST node references into numeric identifiers stored in the state vector, while the `pcIdToNode` list resolves such identifiers back to their corresponding AST nodes during interleaving. In addition, one distinguished ID is reserved for terminal or `null` states in order to represent process termination. Although the concrete numeric values of the assigned IDs may vary between different executions of the compiler, they remain

stable throughout a single exploration run.

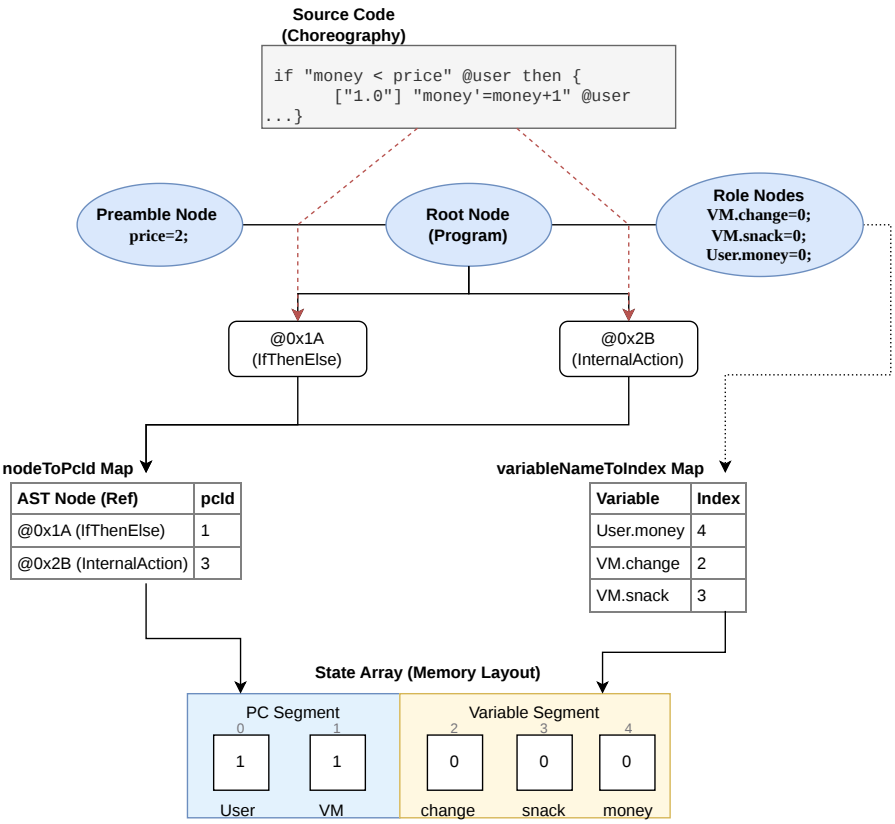


Figure 4.2.: Structure of the initial state vector S_0 for the Vending Machine protocol, illustrating the logical partitioning into the program counter segment and the canonical variable segment.

The PC segment is immediately followed by the variable segment, governed by the `variableNameToIndex` map, which provides storage for all global and local variables. To ensure a canonical state representation, which is vital for efficient duplicate detection, we organize the variables lexicographically by their participating role and variable name. This strict ordering ensures that identical system states always result in identical vector representations, regardless of their discovery order. By determining the `totalVector-`

`Size` as the sum of the PC indices and the variables upfront, we ensure that the vector's layout remains constant throughout the exploration. As illustrated in Figure 4.2, this flat structure allows for $O(1)$ access to any part of the system state, which is essential for the high-performance evaluation of guards and the application of assignments during the state space expansion.

4.3.2. Stable Node Resolution and Guard Evaluation

To optimize the state space exploration, the engine distinguishes between *stable* and *unstable* AST nodes. This distinction is central to our state space reduction strategy. *Unstable nodes* represent administrative or control-flow constructs—such as conditionals (`IfThenElseNode`) or protocol definitions (`RecNode`)—that do not constitute a distinct, observable state of the distributed system. Instead of materializing these nodes as separate entries in the state space, the engine resolves them internally through a *look-ahead* mechanism. Execution only halts, and a new state vector is only materialized, upon reaching a *stable node*. Stable nodes correspond to actual behavioral steps, such as communication statements or internal actions. By skipping purely structural transitions and only capturing stable configurations, the engine ensures that the resulting state space remains compact and focuses exclusively on the meaningful interactions and state changes of the choreography. The core logic of this *look-ahead* mechanism is shown in the simplified Listing 4.3.

```
function resolveToNextStable(node, state):
  current = node
  while current is unstable:
    if current is Protocol or Recursion:
      current = resolveContinuation(current)
    else if current is If-Then-Else:
      if evaluate(current.guard, state):
        current = current.thenBranch
      else if current.hasElseBranch:
      current = current.elseBranch
      else:
        return null // Blocking Guard
  return current // Found stable action or branch
```

Listing 4.3.: Resolution of unstable AST nodes during state space exploration, including the handling of blocking guards via the `null` return value.

In the recursive Vending Machine protocol, exploration starts at the `Start` protocol. At this point, the engine first encounters the guard `money < price`, represented by an unstable `IfThenElseNode`. Since such nodes only determine the control flow, they do not themselves give rise to a transition. Instead, the guard is evaluated against the current state vector. If the condition holds, the engine continues into the corresponding branch and skips over the unstable control-flow nodes until it reaches the next stable node.

In this example, the next stable node is an `InternalActionNode` that increments the `money` variable of the `user`. This action is only executed if the active process is indeed the `user`. If another process is currently considered active, the action is not performed. When the role matches, the variable update is applied and the program counter of the active process is advanced to the next control location.

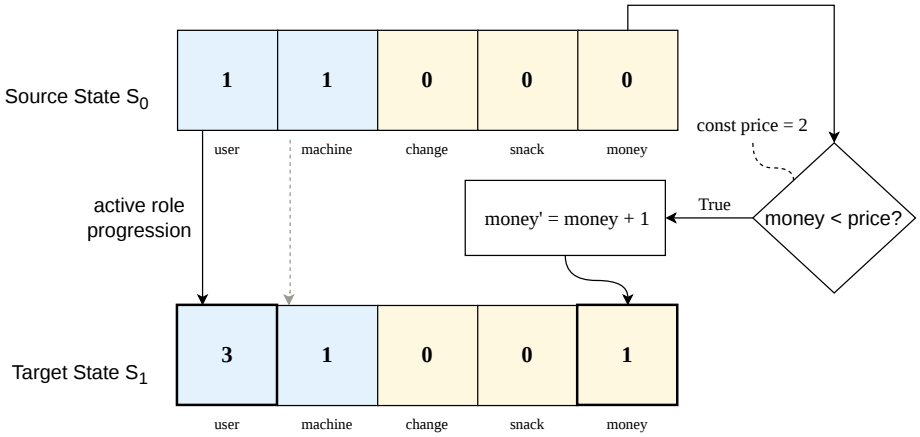


Figure 4.3.: Atomic transformation of the state vector during a variable update step in the `Start` protocol. The engine evaluates the guard using the current variable segment and the global constant `price`. While the active role (`user`) advances to the next stable node (ID$1 \rightarrow 3$) and updates the `money` variable, the passive role (`machine`) remains at its previous control location (ID 1).

As a result, the `user`'s program counter no longer points to the initial guard location, but to the next resolved position in the AST, namely node 3. The `machine`, by contrast, remains at node 1, since it has not yet taken part in the current step. In this way, the successor state reflects both the completed internal action of the active process and the unchanged waiting position of the other role.

Once a stable node has been reached, the engine constructs a successor state on a copy of the current state vector. It applies the required variable updates and advances the program counter of the active process to the next resolved location in the AST. The resulting vector is thus a fully formed successor state corresponding to one executable step of the model, rather than a purely intermediate control-flow configuration.

To construct the state transition graph, every discovered successor is passed to the `TransitionSink` alongside its associated transition rate. This component serves as the 'gatekeeper' of the state space: it performs duplicate detection by checking whether the state vector has already been visited. If the state is new, it is assigned a unique identifier and added to the worklist for further exploration. If the state has been encountered before, the engine simply reuses the existing identifier. By linking the source state to the (new or existing) target state, the `TransitionSink` incrementally builds the complete system model while ensuring that the exploration remains finite and efficient.

In the presence of recursion, this means that cycles arise when exploration returns to a state vector that is already known. Rather than introducing additional intermediate states for structural recursion steps, the implementation records a transition to the existing state, which makes the cyclic structure directly visible in the state space.

A crucial detail in the `resolveToNextStableNode` routine is the handling of unbalanced conditionals. If a guard evaluates to `false` and no `else` branch is defined, the method returns `null` (see Listing 4.3). In the context of the state-space exploration, this `null` value indicates a *blocking guard*: the active process cannot proceed along this execution path in the current state. Consequently, the interleaving engine does not generate any transitions for this process from the given source vector, effectively pruning invalid execution paths and ensuring that the model strictly adheres to the specified synchronization constraints.

4.3.3. Synchronized Interaction and Probabilistic Branching

We realize interactions between roles as synchronized updates of the state vector. When the exploration engine encounters a `BranchNode`, we identify the designated sender and receiver and check whether both processes are ready to participate in the same interaction. We generate a transition only if the receiver is at a compatible control-flow position. More precisely, the receiver's current program counter must either already point to the same `BranchNode` or be able to reach it by resolving only unstable nodes, such as `RecNode`, `ProtocolNode`, or `IfThenElseNode`, without executing any intervening stable action.

This alignment of control flows ensures that communication steps preserve the intended handshake semantics of the choreography. In contrast to internal actions, which affect only the active process, a synchronized interaction updates the control locations of both participating roles and may additionally apply the variable assignments associated with the selected branch.

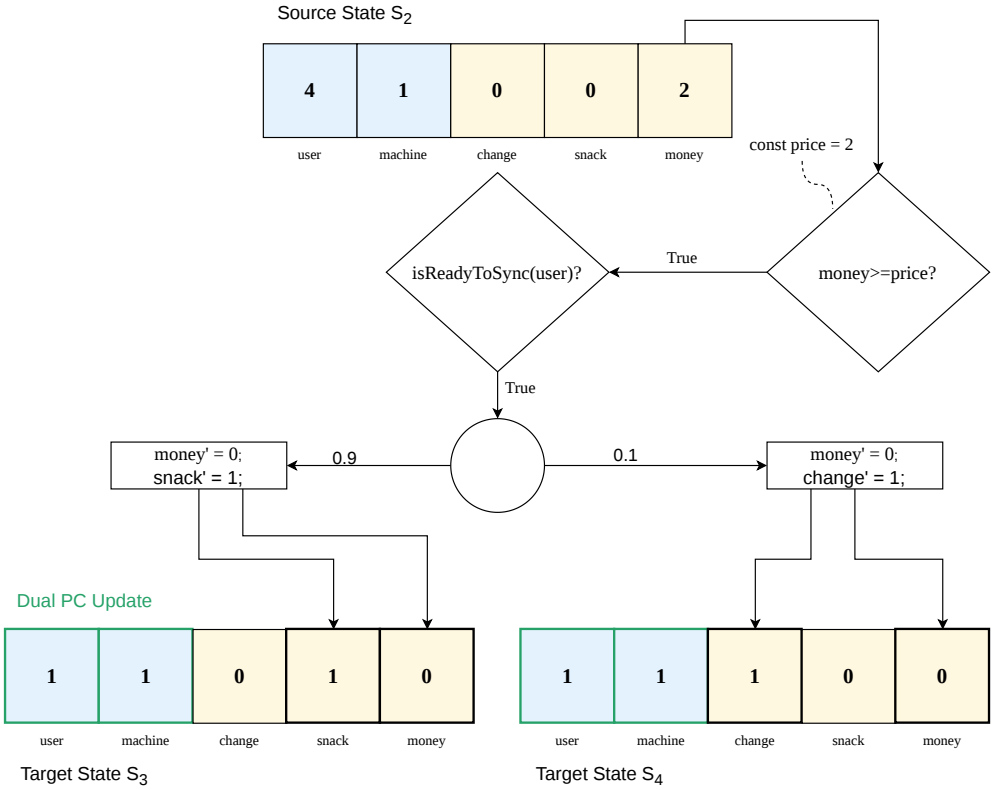


Figure 4.4.: Exploration of a synchronized interaction. Starting from state S_2 , the engine validates the global guard (`money >= price`) and the receiver's readiness (`isReadyToSync`). Upon a successful handshake, the exploration branches into two probabilistic outcomes ($S_3|S_4$) based on the specified rates (0.9 and 0.1). Each branch triggers a *Dual PC Update* (highlighted in green), where the control locations of both participating roles are updated simultaneously alongside the respective variable assignments.

In the recursive `VendingMachine` protocol, this behavior becomes relevant in the `Payout` phase. We can only realize the communication between `machine` and `user` once the accumulated value of `money` satisfies the guard required to leave the `Start` phase. Only then can the `user`'s control flow advance to a position compatible with the `machine`'s branch, thereby enabling the synchronized interaction.

If this compatibility condition is satisfied, we generate one successor for each probabilistic branch of the interaction (see Figure 4.4). Each successor reflects the selected communication outcome by applying the corresponding assignments and updating the program counters of both roles to their respective continuations. In this way, we represent synchronized communication as a single atomic exploration step in the generated state space.

4.3.4. Recursive Control Flow and State-Space Convergence

We realize recursion by redirecting the program counter (PC) to the entry point of the referenced protocol rather than by introducing stack-based calls. Whenever the exploration reaches a `RecNode`, the engine resolves the identifier to its corresponding entry point. The program counter of the active process is then reset to the beginning of the respective protocol, ensuring a clean re-entry into the specified control flow.

This mechanism is directly observable in the payout interaction discussed in the previous section. As shown in Figure 4.4, the target states S_3 and S_4 both feature a program counter of ID 1 for both the `user` and the `machine`. This update is the technical result of the recursive call (`. Start`) at the end of the `Payout` phase. Instead of pushing a return address to a stack, the engine's routine to resolve continuations (`resolveContinuation`) identifies ID 1 as the stable entry point of the `Start` protocol and performs a direct PC-substitution. This approach is sufficient because recursion in our choreographic language does not function as a classical subroutine call that requires returning to a caller's context. Instead, recursive calls represent a transition to a renewed protocol behavior. Since there is no subsequent logic to be executed after a recursion is triggered, there is no need to maintain a call stack or track return addresses.

To ensure convergence for cyclic protocols, we maintain a centralized set of previously discovered state vectors. Before a newly generated successor is explored further, we check whether its vector representation has already been encountered. If it is new, it is inserted into the visited-state set and added to the worklist. Otherwise, the existing state is reused and the corresponding transition closes a cycle in the transition matrix.

This mechanism is essential because recursion may return the control flow to an earlier protocol position without reproducing the same global system state immediately. Even if the program counters assume values that have appeared before, exploration must continue as long as the variable segment differs. For instance, a state in which `snack = 1` is distinct from an otherwise structurally similar state in which `snack = 0`. Convergence is therefore reached only when the complete state vector, including both control locations and variable valuations, matches a previously discovered configuration.

The resulting Discrete-Time Markov Chain (DTMC), shown in Figure 4.5, represents the complete state space of the Vending Machine protocol. To improve readability, the states are color-coded according to their logical role within the protocol. The grey nodes (0–2) denote the initial synchronization and setup phase that is shared by all execution paths. The blue and yellow clusters distinguish different behavioral branches of the choreography, in particular the *Payout* and *Refund* routines. Probabilistic branching points are highlighted by green edges, which correspond to the stochastic choices specified in the ARIA model, such as the 0.9/0.1 distribution used to represent the payout probabilities. Blue edges indicate recursive transitions. These cycles show that the exploration engine is able to close the state space by returning to previously discovered configurations via direct program-counter substitution, thereby yielding a finite exploration despite the recursive structure of the protocol. The corresponding `.sta` and `.tra` files are provided in the Appendix A.2.

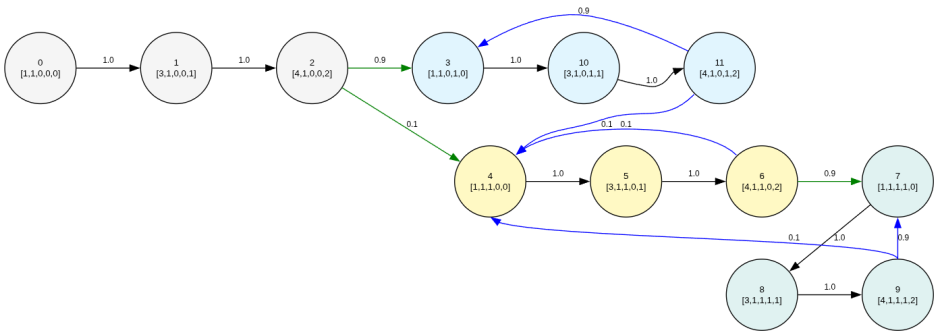


Figure 4.5.: Induced Markov chain of the recursive Vending Machine protocol

4.4. Challenges

While the formal semantics define which states and transitions must be generated, an efficient implementation also depends on how these structures are stored, traversed, and serialized. This section therefore discusses two practical challenges that emerged during the implementation of ARIA: the decoupling of transition generation from file output, and the memory-efficient handling of explicit states during exploration.

4.4.1. Producer–Consumer Decoupling for Transition Output

A practical challenge in explicit-state exploration is the mismatch between CPU-bound successor generation and I/O-bound transition serialization. If every discovered transition were written to disk immediately, the exploration engine would repeatedly block on file output, reducing overall throughput.

To avoid this, we decouple exploration and serialization using a producer–consumer architecture. The exploration engine acts as the producer, while a dedicated background thread consumes transition records and writes them to the output file. Communication between both components is realized via a bounded `LinkedBlockingQueue`.

This design serves two purposes. First, it allows exploration to continue while previously generated transitions are being serialized. Second, the bounded queue limits the amount of buffered output and therefore prevents unbounded memory growth if file output temporarily becomes slower than state-space generation.

To terminate the consumer thread in a controlled manner, we use a sentinel record following the `POISONPILL` [20] pattern. Once exploration is complete and no further transitions remain to be emitted, this special record is inserted into the queue, signaling the consumer to finish processing and close the output stream.

4.4.2. Memory Efficiency and State Deduplication

Since explicit-state exploration is inherently vulnerable to state-space explosion, memory efficiency is a central concern of the implementation. In our setting, each state is represented as an `Object[]` vector containing both control-flow information and variable valuations.

Initially, we attempted to use standard Java collection types, such as `HashSet<>`, for state storage. However, this approach was insufficient because Java arrays rely on identity-based hashing and equality by default. Consequently, logically identical states reconstructed at different points during the exploration were treated as distinct instances, causing the deduplication to fail and leading to redundant exploration and rapid memory exhaustion. To resolve this, we implemented a custom hashing strategy that compares vectors by their actual contents rather than by object identity. This ensures that two independently constructed vectors are recognized as the same logical state whenever both their program-counter segment and their variable segment coincide.

Beyond state storage itself, the order of exploration also affects memory consumption. Our starting point was a standard breadth-first search (BFS) strategy. While BFS is ideal for finding shortest paths to a state, it requires the engine to store the entire frontier of the search tree in the worklist. For concurrent choreographies with high branching factors, this ”frontier explosion” consumed several gigabytes of RAM before any significant depth was reached.

To keep the worklist size manageable, we moved to a biased hybrid search strategy. The worklist is implemented as an `ArrayDeque`, which allows us to alternate between depth-first and breadth-first expansion. In practice, we predominantly explore states in a depth-first manner (DFS), while periodically taking breadth-first steps to maintain coverage across different execution paths. This combination provides a pragmatic balance between exploration depth and memory stability. A purely breadth-first traversal may cause the worklist to grow rapidly, whereas a purely depth-first traversal may delay the discovery of relevant alternative behaviors. By combining both strategies, we obtain a more robust exploration order while keeping the active frontier within manageable bounds.

5. Evaluation

This chapter presents the evaluation of the proposed approach. It first outlines the general evaluation setup and then discusses the selected case studies. For each benchmark, the generated state space and the observed runtime behavior are examined.

5.1. Evaluation Setup

This section describes the general setup of the evaluation. It first discusses the main threats to validity and then outlines the methodology used for the experiments, including the execution environment and measurement procedure.

5.1.1. Threats to Validity

As with any empirical and model-based evaluation, the results presented in this thesis are subject to several threats to validity that may affect their generalizability.

A primary threat is the limited number and representativeness of the evaluated case studies. The evaluation considers only three protocols, namely the Modified ThinkTeam protocol, a Peer-to-Peer protocol, and the Dining Cryptographers protocol. Although these examples were selected to cover relevant characteristics of distributed systems, such as probabilistic behavior, coordination patterns, and race conditions, they represent only a small portion of the broader design space. In particular, other systems may differ substantially in terms of communication topology, synchronization structure, concurrency, cyclic behavior, and overall state-space complexity. As a consequence, the results cannot be assumed to generalize directly to all classes of distributed systems, especially with regard to scalability and performance.

A related threat is selection bias. Since the tool was developed and incrementally refined using these case studies, the implementation may have become partially adapted to their specific characteristics. This creates the risk that the evaluation overestimates the general applicability of the approach.

To mitigate these threats, the selected case studies were chosen to cover different forms of probabilistic distributed behavior and different evaluation concerns, as summarized in Table 5.1. In addition, comparisons with PRISM-generated reference models provide a degree of structural validation. While these measures do not eliminate the limitations discussed above, they increase confidence that the proposed approach is not restricted solely to the concrete examples considered here.

Table 5.1.: Coverage of evaluation case studies in relation to research objectives.

Case Study	**Key Characteristics Covered**	**Primary Evaluation Objective**
Modified ThinkTeam	Cyclic control flow, mutual exclusion, and symbolic rate-based interactions.	Validation of recursive state closure and parametric model generation (CTMCs).
Peer-to-Peer Protocol	Massively concurrent interleavings and combinatorial state-space explosion.	Scalability ”stress test” and structural consistency check against PRISM results.
Dining Cryptographers	Unbalanced `if-then` structures (RQ1) and independent local coin flips (RQ2).	Assessment of semantic resynchronization and complex local branching resolution.

5.1.2. Methodology

The evaluation in this thesis follows a model-based and experimental approach. The selected case studies were used to assess whether the implemented translation is able to generate explicit-state Markov chain models from probabilistic choreographies, that is, models in which states and transitions are represented explicitly. In addition, the evaluation examines the structural properties of the resulting state spaces. Depending

on the example, this includes characteristics such as the number of states, the number of transitions, and the branching structure of the generated model.

All experiments were conducted on the same virtual machine in order to ensure comparability of runtime measurements. The implementation of ARIA was executed using Java 21 on an x86_64 virtual machine running Ubuntu 24.04.4 LTS, providing 8 virtual CPUs and 23 GiB of RAM under KVM virtualization. Since runtime measurements can be influenced by hardware and system configuration, these details are reported to support reproducibility and interpretation of the results.

Because the engine is written in Java, runtime measurements may be affected by the behavior of the Java Virtual Machine (JVM), in particular by just-in-time (JIT) compilation and runtime optimizations. To reduce the influence of startup effects, each benchmark was executed ten times before measurements were recorded. These warm-up runs allow frequently executed code paths to be optimized by the JVM, resulting in more stable and representative timing results.

After the warm-up phase, the actual measurement runs were performed. Each benchmark was executed ten times, and the reported runtimes correspond to the average over these runs. This helps reduce distortion caused by class loading, interpretation overhead, and initial JIT compilation. In general, the measurements include the I/O overhead incurred by writing the generated model data to disk, which may influence the observed runtimes. The main exception is the large Peer-to-Peer benchmark with approximately 33 million states, which was measured without output generation, as serializing the full model exceeded the practical memory limits of the evaluation environment.

5.2. Case Studies

The following case studies were selected to evaluate the proposed approach on different classes of probabilistic distributed systems. They cover characteristic phenomena such as race conditions, repeated retries, independent probabilistic choices, and unbalanced conditional flows, thus providing a diverse basis for assessing the generated models. At the same time, they make it possible to examine synchronization patterns that are closely related to the deadlock-freedom concerns motivating the extended choreography framework.

Collectively, the three case studies provide the basis for answering *RQ 3*. By comparing the generated state spaces and transition counts with established results from the PRISM model

checker, we assess the structural correctness of the ARIA implementation. Furthermore, the performance measurements across varying model sizes demonstrate the practical feasibility of our direct state-space generation approach.

5.2.1. Modified ThinkTeam Protocol

The ThinkTeam Protocol[1] originates from an industrial middleware setting for Product Data Management (PDM) in collaborative CAD environments. Its primary purpose is to manage exclusive access to shared documents through a mechanism called vaulting.

The protocol supports standard operations such as `checkOut` (locking a file for editing) and `checkIn` (releasing the file). A defining characteristic of the original ThinkTeam implementation is its retrial-based access control: the system does not utilize a central waiting list or FIFO queue. If a user attempts to check out a file that is currently locked by another participant, the request is denied. The user must then enter a retrial mode, periodically attempting to gain access until the file becomes available.

Benchmark Objective: Competition, Retrial, and Starvation-Prone Behavior The ThinkTeam case study is a suitable benchmark for our exploration approach because it combines several characteristics that are challenging for explicit-state generation, including concurrent competition, repeated retrials, and cyclic control flow. These features make it a useful test case for assessing whether the implementation can generate the expected transition structure in a systematic and scalable manner.

In a multi-user setting, competition arises whenever the next system state depends on which participant acquires the shared resource first. In the ThinkTeam protocol, this situation occurs whenever a file is released through a `checkIn` operation. At that point, different classes of users may compete for the lock: users that are already in retrial mode after a failed access attempt, and users that initiate a fresh `checkOut` request.

This behavior makes the protocol particularly relevant for the present work. It combines probabilistic branching with repeated contention for a shared resource and therefore induces a cyclic transition structure in which starvation-prone behavior can be represented. As a result, the protocol serves as a compact benchmark for evaluating whether the generated state space faithfully captures the interplay between concurrency, retrial behavior, and probabilistic choice.

[1] https://www.prismmodelchecker.org/casestudies/thinkteam.php

Choreographic Encoding To evaluate the proposed approach, the ThinkTeam protocol was encoded in the extended choreography syntax. Listing 5.1 shows the resulting specification used in the experiments.

```
preamble
"ctmc";
"const int n = 3"
endpreamble;

//Role Definitions
Checkout ->  Checkout : "c = 0";
User[i] -> i in [1...n] User[i] : "u[i] = 0";

{
  InitialAccess := Checkout -> User[i] :

     (+["1 * lambda"] "c'=1" "u[i]'=1" . Success
      +["1 * lambda"] "c'=1" "u[i]'=2" . RetryPhase)

  Success := Checkout -> User[i] : (["1 * mu"] "c'=0" "u[i]'=0" .
  ↪  InitialAccess)

  RetryPhase := Checkout -> User[i] :

       (+["1 * theta"] "c'=1" "u[i]'=1" . Success
        +["1 * theta"] "c'=1" "u[i]'=2" . RetryPhase)

}

```

Listing 5.1.: The modified ThinkTeam protocol in the choreography syntax

The encoding highlights several key features of the choreographic language used in this thesis. The protocol is expressed through three main recursive procedures, namely `InitialAccess`, `Success`, and `RetryPhase`. The status of the shared document is captured by the variable c, where $c = 0$ denotes an unlocked file and $c = 1$ a locked file. The local status of each user i is represented by $u[i]$, where $u[i] = 0$ denotes an idle user, $u[i] = 1$ a user currently holding the file, and $u[i] = 2$ a user in retrial mode. Within this structure, `InitialAccess` models the first checkout attempt and leads either to `Success` or to `RetryPhase`. The `Success` procedure represents the period of document usage followed by its release via `checkIn`, whereas `RetryPhase` captures the repeated competition among users attempting to acquire the lock after a failed access

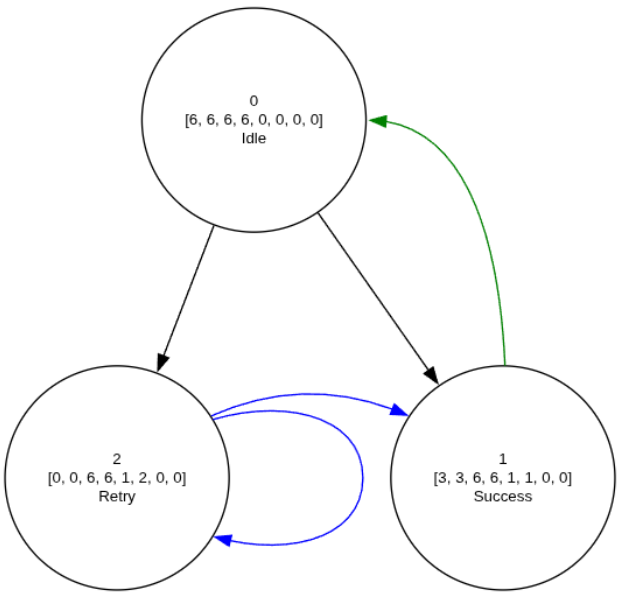


Figure 5.1.: Generated state space for the modified ThinkTeam protocol

attempt.

By systematically interleaving the actions of all n users, the tool constructs a transition structure with symbolic rate annotations that captures the competition between newly arriving requests and users already waiting in retrial mode.

To provide a clearer understanding of the protocol's cyclic logic, Figure 5.1 shows a simplified state-space snippet for a single user $(n = 1)$. While the full state space for larger n (as discussed in Table 5.2) illustrates the scalability of the approach, this reduced view highlights the fundamental transition structure: Black transitions represent initial access attempts (λ), blue transitions retrial attempts (θ), and green transitions document release (μ). The cyclic structure reflects repeated competition for the shared resource under retrial-based access control.

Discussion of the Results The experimental results for the Modified ThinkTeam Protocol, shown in Table 5.2, are consistent with the expected behavior of the ARIA exploration engine and provide empirical support for the correctness of the generated state spaces.

Table 5.2.: Experimental Results for the Modified ThinkTeam Protocol

Users (n)	States	Transitions	Avg. Time [s]
1	3	5	0.0047
2	5	10	0.0049
3	7	15	0.0052
50	101	250	0.0088
100	201	500	0.0132
500	1001	2500	0.1415

The observed linear growth of the state space ($2n + 1$) and the transition count ($5n$) is consistent with the expected theoretical complexity of the protocol. This suggests that the exploration engine handles the interleaving of concurrent request paths in a systematic manner while preserving the intended mutual exclusion behavior of the document vault. The fact that the same structural pattern remains visible even for $n = 500$ roles further indicates that the interaction rules scale robustly for this class of examples.

A central challenge in translating choreographies into explicit state spaces is the consistent management of local instruction pointers. The results are consistent with the assumption that ARIA updates the program counters of both the `Checkout` role and the corresponding `User[i]` processes coherently during interactions. This includes recursive continuations from `Success` back to `InitialAccess` as well as transitions into the `RetryPhase`. Moreover, the absence of unexpected state growth suggests that no spurious intermediate states are introduced during exploration.

Together, these observations suggest that the generated state space provides a suitable basis for later probabilistic analyses once concrete values are assigned to the symbolic rates λ, θ, and μ.

The low runtime, including 0.1415 seconds for 500 users, suggests that for protocols with relatively low coupling, the overhead of exploration remains modest.

5.2.2. Peer-to-Peer Protocol

The Peer-to-Peer (P2P)[2] protocol models a decentralized network in which multiple peers exchange data blocks without a central coordinator. From the perspective of state-space exploration, it constitutes a representative example of a highly concurrent system with substantial interleaving and rapid combinatorial growth.

Benchmark Objective: Interleaving, State-Space Explosion, and Emerging I/O Bottlenecks In contrast to the modified ThinkTeam protocol, which primarily serves as a compact benchmark for cyclic coordination and retrial behavior, the P2P case study is intended to stress the exploration engine under conditions of extreme concurrency. Its main purpose is therefore to evaluate whether the implementation handles the interleaving of many independently progressing participants correctly and whether it remains robust in the presence of severe state-space explosion.

Due to the large number of peers and the relative independence of their local actions, the protocol induces a very high number of reachable configurations. This makes it a suitable benchmark for testing the correctness of the interleaving logic, the robustness of state deduplication, and the practical scalability of the overall exploration procedure.

At the same time, this case study revealed an additional practical limitation during development and testing. While the exploration itself remained efficient, the explicit serialization of the generated transition structure introduced substantial storage demands. The P2P benchmark therefore also became a useful case for observing how, in large-scale explicit-state generation, the bottleneck may shift from computation to output and storage.

Choreographic Encoding As shown in Listing 5.2, the P2P protocol is encoded using a parametric family of `Client[i]` roles, where each client maintains a local state describing which data blocks have already been obtained. Concretely, for every client $i \in \{1, \ldots, n\}$, the variables `b[i]1`, ..., `b[i]5` record the availability of the corresponding blocks, with value 0 indicating that a block is not yet available locally and value 1 indicating that it has been acquired.

The protocol itself is captured by the recursive procedure `PeerToPeer`. In each execution step, a client performs a local probabilistic action that updates one of its block variables and then recurses back to the same protocol. The five branches shown in the listing

[2] https://www.prismmodelchecker.org/casestudies/peer2peer.php

correspond to the acquisition of different blocks, each annotated with its own symbolic rate. This encoding abstracts from concrete message-routing details and instead focuses on the stochastic progression of local block acquisition.

From the perspective of state-space exploration, the crucial aspect of this model is that the clients evolve largely independently. Since each `Client[i]` may perform its next local step without requiring immediate synchronization with the others, the exploration engine must account for a very large number of possible interleavings. Even though the local behavior of an individual client is simple, the combination of parametric roles, recursive continuation, and multiple independently evolving block variables leads to substantial combinatorial growth of the global state space.

This makes the encoding particularly suitable for evaluating whether ARIA can explore large concurrent systems systematically while maintaining a consistent treatment of interleaving and duplicate detection. In this sense, the protocol complements the previous ThinkTeam case study: whereas ThinkTeam emphasizes cyclic coordination and retrial behavior in a comparatively compact state space, the P2P model stresses the exploration engine through massive concurrency and combinatorial growth.

The P2P protocol is the primary benchmark for *RQ 2*. It features a high degree of independence, where multiple peers make local probabilistic decisions about block acquisition. The evaluation focuses on whether the EPP-guided exploration can safely manage the resulting interleavings and the subsequent re-synchronization of these independent local states into a consistent global Markov chain.

Table 5.3.: Comparison of ARIA and PRISM results for the P2P protocol.

n	k	**States (ARIA)**	**States (PRISM)**	**Transitions (ARIA)**	**Transitions (PRISM)**	**Avg. Time [s]**	**States/s**
4	4	65,536	65,536	1,048,576	524,288	2.6451	24,776
4	5	1,048,576	1,048,576	20,971,520	10,485,760	46.5617	22,520
5	4	1,048,576	1,048,576	20,971,520	10,485,760	42.6897	24,564
5	5	33,554,432	33,554,432	838,860,800	419,430,400	2343.3567*	14,319

* Measured without I/O serialization to disk due to memory limits.

```
preamble
"ctmc";
"const int n = 5;"
endpreamble;

Client[i] -> i in [1...n] Client[i] : "b[i]1 = 0", "b[i]2 = 0" , "b[i]3
↪  = 0", "b[i]4 = 0", "b[i]5 = 0";

{
  PeerToPeer := Client[i] -> Client[i] :
   (+["rate1 * 1"] "b[i]1'=1" " " . PeerToPeer
  +["rate2 * 1"] "b[i]2'=1" " " . PeerToPeer
  +["rate3 * 1"] "b[i]3'=1" " " . PeerToPeer
  +["rate4 * 1"] "b[i]4'=1" " " . PeerToPeer
  +["rate5 * 1"] "b[i]5'=1" " " . PeerToPeer)
}

```

Listing 5.2.: The Peer-To-Peer Protocol in the specified syntax

Discussion of the Results A comparison with the corresponding PRISM results (see Table 5.3) reveals that the number of reachable states coincides exactly for all evaluated parameter settings. However, the number of transitions generated by ARIA is consistently twice as large as the PRISM count.

This systematic factor of exactly 2.0 is a direct consequence of how self-loops are handled. The state space of the P2P protocol forms a *binary hypercube* of dimension $D = n \times k$, where n denotes the number of peers and k represents the number of data blocks per peer. The total number of reachable states is thus 2^D.

In our choreographic encoding (see Listing 5.2), each peer provides all k acquisition branches in every state. The ARIA exploration engine exhaustively follows every branch defined in the choreography. If a block is not yet possessed, the assignment results in a state change; if the block is already held, the assignment results in a self-loop. Consequently, every state in the ARIA model has exactly D outgoing transitions, leading to a total transition count of $T_{\text{ARIA}} = D \cdot 2^D$.

In contrast, PRISM's transition count for CTMCs (specifically in `.tra` files) typically excludes self-loops, as they do not affect the exit rates or the transient behavior of the Markov chain. PRISM only records the edges of the hypercube where an actual state change occurs. Since a D-dimensional binary hypercube has exactly $D \cdot 2^{D-1}$ edges, the

resulting ratio is:

$$\frac{T_{\text{ARIA}}}{T_{\text{PRISM}}} = \frac{D \cdot 2^D}{D \cdot 2^{D-1}} = 2 \tag{5.1}$$

This observation is important for two reasons. First, the exact agreement in the number of reachable states ($\Delta S = 0$) provides strong empirical support that both tools explore the identical global configuration space. Second, the systematic factor of two indicates that the difference is structural rather than accidental, reflecting a more *semantically exhaustive* representation of the interaction choices in our model. Taken together, these results suggest that ARIA explores interleavings, independent local decisions, and the resulting re-synchronization behavior in a semantically consistent manner.

The measured throughput remains in the range of approximately 22,000 to 25,000 explored states per second for the smaller configurations and decreases to about 14,000 states per second for the largest instance, reflecting the increasing overhead associated with larger state vectors and transition structures.

The declining throughput for larger benchmark instances is consistent with increasing memory pressure during exploration. As the worklist grows and the number of discovered states increases, the implementation must maintain a larger number of live objects simultaneously, including state vectors, worklist entries, and hash-based indexing structures. A plausible consequence is increased heap pressure and additional garbage-collection overhead, which may contribute to the reduced exploration rate. Although no dedicated heap or garbage-collection profiling was performed, the runtime behavior is consistent with this explanation.

For the largest Peer-to-Peer instances, the limiting factor was not only the size of the reachable state space itself, but also the overhead of serializing and storing the generated model. In particular, the output of explicit transition data imposed substantial I/O demands, which became a practical bottleneck alongside memory consumption.

5.2.3. Dining Cryptographers Protocol

The Dining Cryptographers protocol, originally introduced by Chaum [11], is a classical benchmark for reasoning about anonymity in distributed systems. The protocol describes a group of cryptographers who wish to determine whether their dinner was paid for by their master or by one of themselves, while preserving the anonymity of the actual payer in the latter case.

The protocol proceeds in two phases. First, each cryptographer flips a private coin and shares its outcome only with the neighbour on the right. Second, each participant announces whether the two visible coin values agree or disagree. If a cryptographer is the payer, however, it announces the opposite result. By evaluating the parity of the public announcements, the group can determine whether the master paid or whether one of the cryptographers paid, without revealing the payer's identity.

Benchmark Objective: Guarded Branching and Conditional Control Flow In contrast to the previous case studies, the Dining Cryptographers protocol is particularly useful for evaluating the treatment of conditional control flow. The protocol combines local probabilistic choices with guard-dependent announcements, making it a suitable benchmark for testing whether the exploration engine resolves `IfThenElse` structures correctly during state-space generation.

Of particular interest is the distinction between balanced and unbalanced conditionals. Depending on the current local observations and on whether a participant is the payer, different control-flow paths become enabled. This makes the protocol well suited for assessing whether the implementation handles both branching cases consistently, excludes blocked continuations correctly from the generated state space, and restores a consistent control-flow alignment when a condition evaluates to false and no explicit `else` branch is provided.

This protocol specifically addresses *RQ 1*, as it relies heavily on unbalanced conditional flows to model the cryptographers' announcements. It serves as a test case for the semantic rules developed to handle the absence of explicit `else` branches, ensuring that the exploration engine correctly resynchronizes process control flows even when certain branches are skipped by some participants.

Choreographic Encoding In our encoding (see Appendix A.3), each cryptographer is represented as a separate role with local state variables for the coin value and the final agree/disagree statement. In addition, the model contains a variable identifying the payer. The protocol begins with an initialization step that determines the payer, followed by probabilistic local actions that assign the relevant coin values.

The announcement phase is modeled by guarded transitions whose conditions depend on the local coin value, the neighbouring coin value, and whether the current participant is the payer. As a result, the generated model must combine probabilistic branching, local

Table 5.4.: Comparison of ARIA and PRISM for the Dining Cryptographers benchmark.

N	ARIA			PRISM			Difference	
	States	Transitions	Time (s)	States	Transitions	Time (s)	ΔS	ΔT
3	289	588	0.02999	286	585	0.001	+3	+3
4	1736	4519	0.08096	1733	4580	0.01	+3	-61
5	9879	32158	0.49854	9876	32315	0.03	+3	-157
6	54058	211185	3.21513	54055	211566	0.07	+3	-381

state updates, and dependency-sensitive guarded control flow in a semantically consistent manner.

For a parameterized system of N cryptographers, the protocol generalizes in the standard way: each participant compares its own coin value with that of its neighbour and publishes either agreement or disagreement, with the payer inverting the expected statement. Collectively, these public announcements determine whether the payer was one of the cryptographers or the master.

Discussion of the Results Table 5.4 shows the results of the state-space exploration for the Dining Cryptographers protocol. For all evaluated instances, ARIA constructs exactly three more states than PRISM. This indicates a systematic structural difference between the generated models rather than a random implementation artifact. The number of transitions also differs slightly, although the deviation is not constant across instances: for $N = 3$, ARIA generates three additional transitions, whereas for $N \geq 4$ it generates fewer transitions than PRISM.

Overall, the transition counts indicate a systematic but limited discrepancy between the models generated by ARIA and PRISM. In absolute terms, the deviation grows with N (3, 61, 157, and 381 transitions for $N = 3, \ldots, 6$), while in relative terms it becomes smaller as the models grow. This suggests that the two constructions remain structurally very close, even though they are not derived from fully equivalent specifications.

A plausible explanation for these differences is that the two models do not encode the protocol in exactly the same way. In particular, the PRISM reference model is formulated as an MDP, whereas the ARIA encoding constructs an explicit state space directly from the choreography. Moreover, the control-flow structure differs slightly: the PRISM model uses local status variables together with explicit enabled commands and a synchronizing `done` loop, while the choreographic version resolves progress through sequential phases

and control-flow resynchronization in the exploration semantics. As a result, both models capture the same protocol behavior at a high level, but they need not induce identical state-transition structures.

Despite these structural differences, a comparison with established PRISM reference models remains useful for assessing the plausibility of the models generated by ARIA. As widely used representations of the respective protocols, the PRISM models provide a baseline against which the complexity of the generated state spaces can be interpreted. In particular, comparable growth trends as N increases suggest that the choreographic abstraction does not lead to disproportionate additional state-space growth. Such observations do not establish full equivalence, but they do increase confidence that ARIA produces structurally meaningful models that are broadly consistent with established references.

6. Related Work

This thesis is situated at the intersection of choreographic programming and automated state-space generation for probabilistic verification. In the following, we briefly discuss the most relevant lines of related work and position our contribution with respect to them.

6.1. Choreographic Languages and Frameworks

Montesi's PhD thesis established key foundations of choreographic programming by introducing a formal model and type theory for the paradigm [26]. As part of this research line, he also presented *Chor*, a prototype framework that supports the development of choreography-based programs together with protocol validation and automatic Endpoint Projection (EPP) to executable Jolie code. By enabling developers to specify communication behavior from a single global perspective and verify it against protocol specifications, this approach supports the construction of distributed software that is safe by construction. Montesi's later monograph systematizes these ideas and provides a broader overview of choreographic languages and their theoretical foundations [27].

Building on these foundations, subsequent work proposed languages and frameworks such as *Choral* [19], *HasChor* [35], and *AIOCJ* [34], which explore object-oriented, functional, and adaptive variants of choreographic programming. *Choral* reconstructs choreographic programming through object-oriented abstractions, modelling choreographies as classes and supporting distributed data types, modular composition, and compilation to pure Java libraries. *HasChor*, in contrast, presents a functional formulation in which choreographies are represented as monadic computations in Haskell, with support for higher-order and location-polymorphic choreographies. *AIOCJ* focuses on adaptable distributed applications by allowing designated parts of a choreography to be updated at runtime, while preserving key correctness guarantees such as deadlock freedom and race freedom.

In contrast to existing choreographic programming approaches, which predominantly focus on synthesizing executable distributed implementations from global specifications, this thesis introduced ARIA as a framework centered on formal model construction. Rather than targeting code generation, ARIA supports the automated derivation of analyzable stochastic models from high-level choreographies. In particular, it enables the construction of explicit-state Markov chain models with probabilistic choices and, where applicable, user-provided stochastic parameters. Such models can then be analyzed with probabilistic model checkers such as PRISM in order to study quantitative properties, including performance and reliability. In this way, ARIA extends choreography-based modeling towards quantitative formal analysis.

6.2. Model Generation and State-Space Exploration

Recent work has advanced choreographic programming by mechanizing its theory and compiler infrastructure. Cruz-Filipe et al. formalize the paradigm in *Coq* and provide machine-checked proofs of key correctness properties, including deadlock freedom and the correctness of endpoint projection [14]. Building on this line of work, *hacc* [13] is the first certified compiler for a choreographic language, projecting choreographies from a core calculus to an intermediate process representation and ultimately to Jolie code. Similar efforts, such as *Kalas* [33], likewise target verified end-to-end compilation from choreographies to executable distributed programs.

While these approaches focus on the translation of choreographic specifications into process code, ARIA considers a different direction, namely the construction of explicit-state stochastic models from choreography semantics. In doing so, it follows an on-the-fly explicit-state exploration paradigm comparable to that used in established model checkers such as SPIN [21] and the explicit-state engine of PRISM [22]. Rather than relying on an intermediate process representation, ARIA explores the global state space by iteratively applying the operational semantics of the probabilistic choreography. This design supports the use of state deduplication and worklist-based traversal techniques during model construction.

6.3. Probabilistic Verification of Distributed Systems

The formal analysis of distributed systems through probabilistic model checking has a long tradition, with tools such as PRISM [22] and Storm [16] representing the current standard for the analysis of Discrete-Time and Continuous-Time Markov Chains (DTMCs and CTMCs). In conventional workflows, however, applying such tools typically requires the manual construction of a system model in a dedicated modeling language, such as the PRISM language. This process is often labor-intensive and error-prone, as it requires the modeller to encode the local behavior of each participant while ensuring that the resulting model correctly reflects the intended global synchronization.

To the best of our knowledge, comparatively less attention has been paid to the automated derivation of stochastic models for quantitative analysis from choreographic specifications. In this context, ARIA explores the construction of explicit-state Markov models from probabilistic choreographies, thereby providing a basis for quantitative formal analysis from a global specification.

7. Conclusion and Future Work

This thesis introduced ARIA, a framework for the automated generation of stochastic models from choreographic specifications. By extending choreographic programming with probabilistic branching and, where applicable, stochastic parameters, the framework supports the connection between high-level coordination modeling and quantitative formal analysis.

The evaluation based on representative case studies, including the Modified ThinkTeam protocol and Peer-to-Peer models, suggests that ARIA is capable of constructing explicit-state Markov models for non-trivial distributed systems. Comparisons with PRISM-based reference models indicate that the generated models are structurally meaningful and, for selected cases, show a good degree of agreement with established tool-supported results. A central technical contribution of this work is the implementation of an on-the-fly exploration engine that maintains a global state representation while resolving concurrent interleavings and local program-counter updates during model construction.

Overall, the thesis demonstrates that choreographic specifications can serve not only as a basis for correct-by-construction distributed implementations, but also as a starting point for the automated construction of stochastic models for quantitative formal analysis.

While the current prototype of ARIA demonstrates the feasibility of choreography-based stochastic model generation, several directions for future work remain:

- **Expressiveness and Implementation Support:** While the choreographic paradigm itself is highly expressive, the current prototype implementation of the ARIA engine is restricted to a specific subset of the grammar. A natural next step would be to eliminate these implementation-specific limitations by expanding the engine's capability to resolve additional constructs, such as collective synchronization and more generalized guarded command patterns. Supporting a broader fragment of the grammar would bridge the gap between the theoretical framework and the

tool's practical expressiveness, enabling the modeling of a wider range of complex distributed protocols.

- **Implementation Scalability and Optimization:** The performance analysis of our prototype reveals that for large-scale models, practical bottlenecks are increasingly driven by implementation-specific factors, such as JVM memory management and the high I/O overhead of model serialization. Future work should therefore focus on technical optimizations of the ARIA backend, specifically by investigating more memory-efficient internal data structures and compact binary export formats. Furthermore, the integration of algorithmic reduction techniques—such as partial-order or symmetry reduction—directly into the exploration engine could significantly increase the tool's capacity to handle the state-space explosion in complex distributed systems.
- **Formal Semantics and Correctness Guarantees:** Another important direction is the formal study of the semantic relationship between source choreographies and the generated stochastic models. Establishing a bisimulation result, or a similar correspondence between the operational semantics of the choreography and the induced transition system, would strengthen the theoretical foundations of the approach.
- **Testing and Validation Infrastructure:** As the framework matures, its implementation could be strengthened through a more systematic testing strategy. Unit tests for core components such as parsing, semantic rule application, and state deduplication could be complemented by integration tests covering the full pipeline from choreography specification to exported model. Such a test suite would improve confidence in future language extensions and help detect regressions during further development.

Among the directions outlined above, a formal semantic correspondence between the choreographic source language and the generated stochastic models appears particularly important. While scalability and language expressiveness are relevant for the practical development of ARIA, the broader motivation of choreographic programming lies in the close connection between high-level specifications and their correct realizations. In the context of stochastic model generation, this suggests the need to complement empirical evaluation with a more rigorous formal account of the translation. For example, establishing a bisimulation result, or a similar semantic correspondence, would provide stronger justification that the transitions of the generated Markov model faithfully reflect the coordination behavior described by the source choreography. Such a result would not

only strengthen the theoretical foundations of ARIA, but also increase confidence in the quantitative analyses carried out on the generated models.

Bibliography

[1] Ali Ali and Lindsey Kuper. *Communicating Chorrectly with a Choreography*. Educational Zine. Supported by the National Science Foundation (NSF) Grant No. CCF-2145367. 2023. URL: https://lsandko.github.io/communicating-chorrectly/.

[2] Michael Armbrust et al. "A view of cloud computing". In: *Commun. ACM* 53.4 (2010), pp. 50–58. URL: https://doi.org/10.1145/1721654.1721672.

[3] Luigi Atzori, Antonio Iera, and Giacomo Morabito. "The Internet of Things: A survey". In: *Comput. Networks* 54.15 (2010), pp. 2787–2805. URL: https://doi.org/10.1016/j.comnet.2010.05.010.

[4] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. The MIT Press, 2008. ISBN: 978-0-262-02649-9.

[5] Francesco Adalberto Bianchi, Alessandro Margara, and Mauro Pezze. " A Survey of Recent Trends in Testing Concurrent Software Systems ". In: *IEEE Transactions on Software Engineering* 44.08 (Aug. 2018), pp. 747–783. ISSN: 1939-3520. URL: https://doi.ieeecomputersociety.org/10.1109/TSE.2017.2707089.

[6] Francesco Adalberto Bianchi, Alessandro Margara, and Mauro Pezzè. "A Survey of Recent Trends in Testing Concurrent Software Systems". In: *IEEE Transactions on Software Engineering* 44.8 (2018), pp. 747–783. URL: https://doi.org/10.1109/TSE.2017.2707089.

[7] Robert T. Braden. *Requirements for Internet Hosts - Communication Layers*. RFC 1122. Oct. 1989. URL: https://www.rfc-editor.org/info/rfc1122.

[8] Marco Carbone and Fabrizio Montesi. "Deadlock-freedom-by-design: multiparty asynchronous global programming". In: *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*. Ed. by Roberto Giacobazzi and Radhia Cousot. ACM, 2013, pp. 263–274. URL: https://doi.org/10.1145/2429069.2429101.

[9] Marco Carbone and Fabrizio Montesi. "Merging Multiparty Protocols in Multiparty Choreographies". In: *Proceedings Fifth Workshop on Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES 2012, Tallinn, Estonia, 31 March 2012*. Ed. by Simon J. Gay and Paul Kelly. EPTCS. 2012, pp. 21–27. URL: https://doi.org/10.4204/EPTCS.109.4.

[10] Marco Carbone and Adele Veschetti. "A Probabilistic Choreography Language for PRISM". In: *Coordination Models and Languages: 26th IFIP WG 6.1 International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024, Groningen, The Netherlands, June 17–21, 2024, Proceedings*. Groningen, The Netherlands: Springer-Verlag, 2024, pp. 20–37. ISBN: 978-3-031-62696-8. URL: https://doi.org/10.1007/978-3-031-62697-5_2.

[11] David Chaum. "The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability". In: *Journal of Cryptology* 1.1 (1988), pp. 65–75.

[12] E. G. Coffman, M. Elphick, and A. Shoshani. "System Deadlocks". In: *ACM Comput. Surv.* 3.2 (June 1971), pp. 67–78. ISSN: 0360-0300. URL: https://doi.org/10.1145/356586.356588.

[13] Luís Cruz-Filipe, Lovro Lugović, and Fabrizio Montesi. *Certified Compilation of Choreographies with hacc*. 2023. arXiv: 2303.03972 [cs.PL]. URL: https://arxiv.org/abs/2303.03972.

[14] Luís Cruz-Filipe, Fabrizio Montesi, and Marco Peressotti. "Formalising a Turing-Complete Choreographic Language in Coq". In: *12th International Conference on Interactive Theorem Proving, ITP 2021, Rome, Italy (Virtual Conference), June 29 - July 1, 2021*. Ed. by Liron Cohen and Cezary Kaliszyk. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 15:1–15:18. URL: https://doi.org/10.4230/LIPIcs.ITP.2021.15.

[15] Luís Cruz-Filipe et al. "Modular Choreographies: Bridging Alice and Bob Notation to Java". In: *Post-proceedings of Microservices 2020/2022*. 2022.

[16] Christian Dehnert et al. "A Storm is Coming: A Modern Probabilistic Model Checker". In: *Computer Aided Verification (CAV), Part II*. Vol. 10427. Lecture Notes in Computer Science. Springer, 2017, pp. 592–600.

[17] Nicola Dragoni et al. "Microservices: Yesterday, Today, and Tomorrow". In: *Present and Ulterior Software Engineering*. Ed. by Manuel Mazzara and Bertrand Meyer. Cham: Springer International Publishing, 2017, pp. 195–216. ISBN: 978-3-319-67425-4. URL: https://doi.org/10.1007/978-3-319-67425-4_12.

[18] Saverio Giallorenzo, Fabrizio Montesi, and Maurizio Gabbrielli. “Applied Choreographies”. In: *Formal Techniques for Distributed Objects, Components, and Systems*. Ed. by Christel Baier and Luís Caires. Cham: Springer International Publishing, 2018, pp. 21–40. ISBN: 978-3-319-92612-4. URL: https://doi.org/10.1007/978-3-319-92612-4%5C_2.

[19] Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti. “Choral: Object-oriented Choreographic Programming”. In: *ACM Trans. Program. Lang. Syst.* 46.1 (2024), 1:1–1:59. URL: https://doi.org/10.1145/3632398.

[20] Brian Goetz et al. *Java Concurrency in Practice*. Section 7.2.2. Addison-Wesley Professional, 2006.

[21] Gerard J. Holzmann. *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley, 2003. ISBN: 0-321-22862-6.

[22] M. Kwiatkowska, G. Norman, and D. Parker. “PRISM 4.0: Verification of Probabilistic Real-time Systems”. In: *Proc. 23rd International Conference on Computer Aided Verification (CAV'11)*. Ed. by G. Gopalakrishnan and S. Qadeer. Vol. 6806. LNCS. Springer, 2011, pp. 585–591.

[23] Marta Kwiatkowska, Gethin Norman, and David Parker. “Probabilistic Model Checking: Advances and Applications”. In: *Formal System Verification: State-of the-Art and Future Trends*. Ed. by Rolf Drechsler. Cham: Springer International Publishing, 2018, pp. 73–121. ISBN: 978-3-319-57685-5. URL: https://doi.org/10.1007/978-3-319-57685-5_3.

[24] Leslie Lamport. “Time, clocks, and the ordering of events in a distributed system”. In: *Concurrency: The Works of Leslie Lamport*. New York, NY, USA: Association for Computing Machinery, 2019, pp. 179–196. ISBN: 9781450372701. URL: https://doi.org/10.1145/3335772.3335934.

[25] Charles E. McDowell and David P. Helmbold. “Debugging concurrent programs”. In: *ACM Comput. Surv.* 21.4 (Dec. 1989), pp. 593–622. ISSN: 0360-0300. URL: https://doi.org/10.1145/76894.76897.

[26] Fabrizio Montesi. *Choreographic Programming*. English. 2013.

[27] Fabrizio Montesi. *Introduction to Choreographies*. Cambridge: Cambridge University Press, 2023. ISBN: 978-1009221125.

[28] Madanlal Musuvathi et al. “Finding and Reproducing Heisenbugs in Concurrent Programs”. In: Jan. 2008, pp. 267–280.

[29] Roger M. Needham and Michael D. Schroeder. "Using encryption for authentication in large networks of computers". In: *Commun. ACM* 21.12 (Dec. 1978), pp. 993–999. ISSN: 0001-0782. URL: https://doi.org/10.1145/359657.359659.

[30] Robert Netzer and Barton Miller. "What are Race Conditions? - Some Issues and Formalizations". In: *ACM letters on programming languages and systems* 1 (Sept. 1992).

[31] J. R. Norris. *Markov Chains*. Cambridge University Press, 1997.

[32] Terence Parr. *ANTLR (ANother Tool for Language Recognition)*. https://www.antlr.org/, accessed 2024-03-05. 2024.

[33] Johannes Åman Pohjola et al. "Kalas: A Verified, End-To-End Compiler for a Choreographic Language". In: *13th International Conference on Interactive Theorem Proving (ITP 2022)*. Ed. by June Andronick and Leonardo de Moura. Vol. 237. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 27:1–27:18. ISBN: 978-3-95977-252-5. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2022.27.

[34] Mila Dalla Preda et al. "Dynamic Choreographies: Theory And Implementation". In: *Logical Methods in Computer Science* Volume 13, Issue 2, 1 (Apr. 2017). ISSN: 1860-5974. URL: https://lmcs.episciences.org/3263.

[35] Gan Shen, Shun Kashiwa, and Lindsey Kuper. "HasChor: Functional Choreographic Programming for All (Functional Pearl)". In: *Proceedings of the ACM on Programming Languages* 7.ICFP (Aug. 2023), pp. 541–565. ISSN: 2475-1421. URL: http://dx.doi.org/10.1145/3607849.

[36] Maarten van Steen and Andrew S. Tanenbaum. "A brief introduction to distributed systems". In: *Computing* 98.10 (2016), pp. 967–1009. URL: https://doi.org/10.1007/s00607-016-0508-7.

A. Appendix

A.1. Grammar

```
protocol : (preamble)? (varDef)? SEMICOLON (roleDef)+  CLPAR (protocolID ASSIGN blockStatement)* CRPAR ;

protocolID : ID ;

blockStatement : (statement)+;

statement : branch | ifThenElse | end | internalAction | rec;

branch : inputRole=role FROM outputRole+=role (COMMA outputRole+=role)* COLON LPAR (branchStat* | commStat )RPAR ;

branchStat : (BRANCH SLPAR rateValues+=rate SRPAR updates DOT statement) ;

commStat : (SLPAR rateValues+=rate SRPAR updates DOT statement) ;

ifThenElse : IF cond AT role (COMMA cond AT role)* THEN CLPAR thenStat=statement CRPAR (ELSE CLPAR elseStat=statement CRPAR)?;

rec : protocolID ;

end : END (AT CLPAR roles+=role (COMMA roles+=role)* CRPAR)?;

internalAction : SLPAR rate SRPAR message AT role DOT statement;

updates : (SQLPAR (prec=actions) SQRPAR)? upds=message ;

message : actions | first=DOUBLE_STRING loop | loop second=DOUBLE_STRING | loop*;
```

```
27
28  actions : (action+=DOUBLE_STRING)? ( action+=DOUBLE_STRING)*  ;
29
30  loop : FOREACH LPAR indexIteration=index op=(EQ | LE | GE | LEQ | GEQ |
↪  NEQ ) upperBound=index RPAR DOUBLE_STRING AT role;
31
32  preamble : PREAMBLE (modelType SEMICOLON)? (variableDecl)* ENDPREAMBLE ;
33
34  varDef : CHAR EQ INTEGER;
35
36  modelType : DOUBLE_STRING;
37
38  roleDef : role FROM (indexSpec)? roleSpec* SEMICOLON;
39
40  roleSpec : (role (COLON (roleVar (COMMA roleVar)*))?);
41
42  roleGroup : ID  ;
43
44  roleIndex : ID SLPAR (index | (index BRANCH INTEGER)) SRPAR ;
45
46  indexSpec : index IN SLPAR INTEGER DOTS upperBound=CHAR SRPAR ;
47
48  role : (roleGroup | roleIndex) ;
49
50  roleVar : DOUBLE_STRING;
51
52  variableDecl : DOUBLE_STRING;
53
54  rate : DOUBLE_STRING ;
55
56  cond : DOUBLE_STRING ;
57
58  index : CHAR ;
59
60  SEMICOLON   : ';' ;
61  COLON     : ':' ;
62  DOT     : '.' ;
63  DOTS     : '...' ;
64  COMMA    : ',' ;
65  BRANCH     : '+' ;
66  FROM     : '->';
67  ASSIGN     : ':=' ;
68  NEQ     : '!=' ;
69  EQ       : '=' ;
70  LEQ      : '<=';
71  GEQ      : '>=';
```

```
LE       : '<';
GE       : '>';
UNDERSCORE    : '_' ;
STAR      : '*' ;
LPAR        : '(' ;
RPAR        : ')' ;
SLPAR      : '[' ;
SRPAR      : ']' ;
CLPAR       : '{' ;
  CRPAR       : '}' ;
SQLPAR     : '<<' ;
SQRPAR     : '>>' ;
AT        : '@' ;
IF        : 'if';
THEN     : 'then';
ELSE     : 'else';
IFplus     : 'IF' ;
ELSEplus  : 'ELSE';
END      : 'END';
AND       : '&&' ;
PREAMBLE  : 'preamble';
ENDPREAMBLE : 'endpreamble';
CONST      : 'const';
FOREACH     : 'foreach';
IN       : 'in' ;
WS       : [ \t\r\n\u00a0]+ -> skip ;
DOUBLE_STRING : '"' ~('"')+ '"';

//Numbers
fragment DIGIT : '0'..'9';
INTEGER        : DIGIT+;

//IDs
CHAR  : 'a'..'z' |'A'..'Z' ;
ID               :  (CHAR | UNDERSCORE)+ (CHAR | DIGIT | UNDERSCORE |
↪  STAR)* CHAR*;

```

Listing A.1.: Grammar for ARIA

A.2. Vending Machine: `.sta` and `.tra` files

State Vectors

```
0: [1, 1, 0, 0, 0]
1: [3, 1, 0, 0, 1]
2: [4, 1, 0, 0, 2]
3: [1, 1, 0, 1, 0]
4: [1, 1, 1, 0, 0]
5: [3, 1, 1, 0, 1]
6: [4, 1, 1, 0, 2]
7: [1, 1, 1, 1, 0]
8: [3, 1, 1, 1, 1]
9: [4, 1, 1, 1, 2]
10: [3, 1, 0, 1, 1]
11: [4, 1, 0, 1, 2]
```

Transitions

```
12 16
0 1 1.0
1 2 1.0
2 3 0.9
2 4 0.1
3 10 1.0
4 5 1.0
5 6 1.0
6 4 0.1
6 7 0.9
7 8 1.0
8 9 1.0
9 4 0.1
9 7 0.9
10 11 1.0
11 3 0.9
11 4 0.1
```

Listing A.2.: State vector mapping (`.sta`, left) and the resulting transition matrix (`.tra`, right) for the VendingMachine example

A.3. The Dining Cryptographers Protocol

```
preamble
"dtmc";
"const int N = 3;"
"const int p1 = 1;" "const int p2 = 2;" "const int p3 = 3;"
endpreamble;

crypt[i] -> i in [1...N] crypt[i] : "coin[i] = 0", "agree[i] = 0",
↪  "pay=0";

{InitPay :=
  if "i = 1" @crypt[i] then {
    if "pay = 0" @crypt[i] then {
      crypt[i] -> crypt[i] : (+ ["0.25"] "pay'=0" . Crypto
      + ["0.25"] "pay'=1" . Crypto
      + ["0.25"] "pay'=2" . Crypto
      + ["0.25"] "pay'=3" . Crypto)
    } else { Crypto }
  } else {
    if "pay > 0" @crypt[i] then { Crypto }}

  Crypto := if "coin[i] = 0" @crypt[i]
  then { crypt[i] -> crypt[i] :
     (+ ["0.5"] "coin[i]' = 1" . Crypto2
    + ["0.5"] "coin[i]' = 2" . Crypto2)}

  Crypto2 := if "coin[i]>0 && coin[i+1]>0" @crypt[i] then {
    if "coin[i] = coin[i+1]" @crypt[i] then {
      if "pay=p[i]" @crypt[i] then {
        ["1.0"] "agree[i]'=0" @crypt[i] . Crypto3     } else {
        ["1.0"] "agree[i]'=1" @crypt[i] . Crypto3 }}else {
      if "pay=p[i]" @crypt[i] then {
        ["1.0"] "agree[i]'=1" @crypt[i] . Crypto3
      } else {
        ["1.0"] "agree[i]'=0" @crypt[i] . Crypto3
  }}}

  Crypto3 := END}
```

Listing A.3.: The Dining Cryptographers Protocol in the specified syntax

B. Appendix

Erklärung zur Nutzung von KI-Hilfsmitteln

Ich erkläre hiermit, dass ich für die Erstellung der vorliegenden Arbeit KI-basierte Werkzeuge zur Unterstützung verwendet habe.
Die Werkzeuge wurden für folgende Aufgaben eingesetzt:

- Unterstützung bei der sprachlichen Überarbeitung und Korrektur.
- Unterstützung bei der Erstellung von Tabellen und Diagrammen
- Unterstützung bei der Fehlersuche im Programmcode.

Ich bestätige, dass die KI-generierten Inhalte kritisch geprüft, überarbeitet und in die eigene gedankliche Leistung integriert wurden. Die Verantwortung für die Richtigkeit und wissenschaftliche Integrität der Arbeit liegt allein bei mir.

Darmstadt, den April 14, 2026

(Unterschrift)